

基于多目标优化的生产企业原材料订购与转运方案研究

摘要

为保障生产企业持续稳定发展,本文建立了综合评价模型帮助企业筛选高质量合作供应商,并根据企业生产需求和供应商、转运商实际情况制定未来半年的订购和运输计划。同时展望未来,对相同原材料市场下企业产能最大化提出见解。

针对问题一,通过对 402 家供应商的供货特征进行量化分析,本文初步确定了供应次数总和、供应量总和、订货总量、单次最大供货量和平均供货量共五个核心指标。引入有效供货周数据计算的供货量均方误差(供货稳定性),体现供应商的兑现能力。在基于 **Spearman 相关性**分析剔除共线性指标后,确定供货规模占比、供货完成率、供货稳定性和物料集中度四项评价指标,采用**熵权法**客观赋权结合 **TOPSIS 法**构建综合评价模型,筛选得到 50 家关键供应商,为后续订购方案优化奠定基础。

针对问题二,第一小问为满足产能的最少供应商问题,先将供应商供货能力折算为等效成品产能,建立 **0-1 整数规划模型**,通过**贪心累加**求解得到满足每周产能的最少供应商数量为 **28 家**;第二小问,在此基础上以采购、库存、损耗综合成本最小化为目标,纳入库存动态平衡、安全库存、供应商供货上限、转运商运力与指派唯一性等约束,构建**混合整数线性规划模型**。求解得到 24 周成本最优订购转运方案,方案总成本为 **493999.08 元**,全程满足安全库存要求;最后从库存运行状态、成本开销、供应商订货结构三个方面进行分析,对实施效果进行综合评价,该套订货转运方案在 24 周规划周期内严格遵守安全库存约束,能够有效规避生产断供风险。

针对问题三,面对“多采购 A 类、少采购 C 类”的原料结构调整需求,首先设置 5 组不同偏好强度的约束场景,再采用**熵权法**计算指标权重,并针对损耗率在高 A 占比区间区分度不足的问题进行权重修正。其次构建**加权多目标混合整数规划模型**,求解得到兼顾成本、原料结构与转运损耗的综合方案。结果显示方案实现 A 类采购占比 32%、C 类占比 28%,转运损耗率降至 0.02%,相比基准方案总成本上涨 35.7%;在权重±20%扰动下的**灵敏度分析**验证了方案具备良好稳健性。

针对问题四,技术改造后的产能提升问题,基于现有供应商与转运商资源,分品类核算原料供应上限,引入**运力动态优化分配机制**,构建产能最大化模型,结合瓶颈识别产能制约因素。结果表明,现有供应商组合下企业周最大产能可达 **29926.55 立方米**,较原产能提升 **6.12%**,产能提升的核心瓶颈为供应商供货能力,可扩展供应商边际增量趋近于零,无需扩展,转运运力存在富余,带入 MILP 模型,求解得到产能提升后对应的 24 周订购转运方案,方案总成本为 **509113.46 元**,全周期满足安全库存约束和转运商运输能力。

关键词: **熵权-TOPSIS 组合模型** **0-1 整数规划模型** **贪心累加求法**
混合整数线性规划模型 **运力动态优化分配机制**

一、问题重述

1.1 问题背景

某建筑和装饰板材生产企业生产所需原材料分为 A、B、C 三类。A 类、B 类原材料采购单价分别较 C 类高 20%、10%，三类原材料的运输、储存单位费用相同。企业全年按照 48 周组织生产，需要提前制定未来 24 周原材料订购与转运计划。

企业每周产能上限为 2.82 万立方米。每生产 1 立方米成品，需要消耗 A 类原材料 0.6 立方米，或 B 类原材料 0.66 立方米，或 C 类原材料 0.72 立方米。供应商实际供货量与企业订货量存在偏差，企业会全部收购供应商的实际供货。为维持连续生产，要求原材料库存不得低于满足两周生产消耗的物料数量。

原材料经转运商运输会产生损耗，扣除损耗后运抵企业仓库的物料称为接收量。每家转运商每周最大运输能力为 6000 立方米，要求同一供应商每周的供货尽量由同一家转运商完成转运。

1.2 问题提出

现有附件 1 记录企业近 5 年 402 家供应商的订货量与供货量数据，附件 2 记录 8 家转运商的历史运输损耗率数据。建立数学模型，为企业设计合理的原材料订购与转运方案。

问题 1：根据附件 1 数据对 402 家供应商的供货特征开展量化分析，构建评价供应商对企业生产保障重要程度的数学模型，筛选得到 50 家最重要的供应商。

问题 2：第一小问，依据问题 1 的分析结果，求解能够满足企业生产需求的最少供应商数量；第二小问，以企业经济成本最低为目标，制定未来 24 周每周原材料订购方案；再以此订购方案为基础，制定总损耗最小的转运方案，并对订购、转运两套方案开展实施效果分析。

问题 3：企业调整采购策略，希望尽可能多采购 A 类原材料、尽可能少采购 C 类原材料，以此压缩转运仓储成本，同时追求更低的转运损耗。在该目标下设计对应的 24 周订购方案与转运方案，并分析方案的实施效果。

问题 4：企业具备提升产能的能力，供应商与转运商资源维持不变。结合现有供给与转运约束，确定企业每周产能最大提升幅度，并给出产能提升后未来 24 周的原材料订购及转运方案。

二、问题分析

2.1 对问题一的分析

本文通过对附件 1 中 402 家供货商的供货特征进行量化处理，确定了供应次数、供货总量、订货总量、单次最大供货量和平均供货量五个核心指标，引入供货完成率表征供应商的兑现水平，为后续构建评价模型筛选关键供应商做铺垫。再用 Spearman 相关性分析剔除供货次数总和，最后确定四项评价指标为供货规模占比、供货完成率、供货稳定性和物料集中度。对筛选后的指标构建原始决策矩阵，采用极差归一化消除各指标量纲差异。借助熵权法计算各项指标客观权重。将标准化矩阵与权重结合得到加权矩阵，使用 TOPSIS 方法计算每家供应商的综

合贴近度，将全部供应商按照贴近度降序排列，选取排名靠前的 50 家作为关键供应商。

2.2 对问题二的分析

第一小问：参考问题一要求至少选择多少家供应商才可能满足需求，相当于在保障企业每周 2.82 万立方米成品的产能下，选取数量最少的供应商组合。本文将问题一中的 50 家供应商作为研究对象集合，将供应商的原材料供应能力折算为等效成品产能，建立 0-1 整数规划模型，满足产能覆盖约束（供应商等效成品产能总和恰好大于 2.82 万 m^3 ），从而得出能够满足企业生产需要的最少的供应商数量。

第二小问：根据第一小问的模型确定出每周的原材料供应商（共 24 周），再对这些供应商制定未来 24 周每周经济成本最低的订购方案，建立混合整数线性规划模型^[1]，以库存动态平衡、安全库存（周期末库存 $\geq$ 两周原料需求量）供应商供货上限和转运业务（单供应商单周仅由一家转运商运输）为约束条件，最后得出 24 周的订货量、转运指派以及期末库存的全套最优方案。

第三小问：结合库存运行状态，成本开销和供应商订货结构对最终实施效果进行综合评价分析。

2.3 对问题三的分析

基于问题二的决策变量，以多采购 a 类少采购 c 类原材料、减少转运商的损耗率以及减少仓储与运输成本为优化目标进行分析。首先设计 A、C 两类原材料占比约束场景，分别求解得到 5 个场景的方案，再利用熵权法从备选方案数据中客观计算其目标权重，进而建立加权多目标 MILP 模型。并对其结果进行灵敏度分析。

2.4 对问题四的分析

以未来 24 周的产能为决策变量，以最大化未来 24 周产能提高率均值为优化目标，以转运原材料大于 0、每家转运商运输量不超过 6000 立方米、一家供应商每周供应的原材料尽量由一家转运商运输且任一周的库存量为自然数作为约束条件建立混合整数线性规划模型。

三、模型假设

- 假设 1：该企业在过去五年内的生产规模相对稳定，未出现较大变化。
- 假设 2：该企业的仓库有不少于满足两周生产需求的原材料库存量。
- 假设 3：无极端天气导致转运商在转运过程中原材料的大量损耗。
- 假设 4：各转运商的运输损耗率相互独立。
- 假设 5：企业生产过程中 A、B、C 三类原材料可按任意比例混合使用，不受生产工艺配方限制，各类原材料之间不存在替代约束或配比下限要求。

四、符号说明

符号	说明
p_{ij}	第 j 个指标下第 i 个供应商的占比
e_j	第 j 个指标的熵值

w_j	第 j 个指标的权重
C_i	相对贴进度（TOPSIS 得分）
μ_i	有订单时的历史平均周供货量
E_i	等效成品产能
$r_m(i)$	成品消耗系数
α_m	第 m 类原料在成品生产中的固定用量比例
D_m	每周原料消耗量
I_t	周期期末库存
Cap_m^{supply}	原料最大周供应能力
P_m^{supply}	成品产能上限

五、模型的建立与求解

5.1 问题一

5.1.1 量化分析供货商供货特征

（1）考虑到企业实际生产中，既需要供应商长期稳定供货，又需应对突发需求波动，单纯依赖总量或次数均不足以全面刻画供应商对该企业生产的价值。对附件 1 中近 5 年的订货与供货历史数据进行处理，得到供应次数、供货总量、订货总量、最大单次供货量和平均供货量五个互补性指标开展量化分析。

供应次数总和（供货频次）：统计附件 1 中供应商供货量大于 0 的周数，大于 0 的周数记为 1 次，否则不计入。表征供应商能够实际提供原材料的频次，反映供应商供货的持续能力。

b.供应量总和：对统计周期内每周实际供货量进行累加，企业需要源源不断地获取原材料用来生产，体现供应商的原材料供给的总体规模。

c.订货总量：企业向该供应商下达的累计订货数量，反映出企业历史阶段的订货情况。

d.最大单次供货量：由于该企业对供货商实际提供的原材料总是全部收购，供货商为实现利益最大化，一定会保持最大生产效率。因此取供应商全部历史周供货量的最大值，代表该供应商可以达到的单周供给能力上限。

e.平均供货量：该指标等于供应量总和除以供应次数总和，也能在一定程度上反映该供应商的供货能力，部分供应商偶尔大批供货但平常供给薄弱，平均供货量可反应这些供应商的持续输出能力。

本文引入供货完成率，定义为统计周期内供应商累计实际供货量与企业累计向供应商下达订货量的比值，是衡量供应商订单兑现能力的核心指标。供货完成率把企业需求量与供应商供货量这两个核心数据直接进行匹配，能够直观地反应供应商能否兑现合作承诺、满足企业的真实生产需求。根据供货完成率可为企业筛选出提供稳定保障的供应商，为后续的订购方案优化提供支撑。

使用 python 计算出供货完成率，结果如下表所示。（表格按照供货量总和降序排列；因篇幅原因，选取前 15 家供应商，其余数据见附录）

表 1 402 家供应商供货特征与供货完成率计算结果表

供应商 ID	材料分类	供应次数总和	供货量总和	订货量总和	单次最大供货量	平均供货量	供货完成率
S229	A	240	354887	359885	3147	1478.696	0.986112
S361	C	240	328080	333452	2816	1367	0.98389
S140	B	219	302047	481103	21293	1258.529	0.627822
S108	B	240	240950	271445	7885	1003.958	0.887657
S151	C	240	194498	266510	21267	810.4083	0.729796
S340	B	240	171426	172000	1181	714.275	0.996663
S282	A	240	169340	168531	1724	705.5833	1.0048
S275	A	240	158553	158150	966	660.6375	1.002548
S329	A	240	156518	156350	971	652.1583	1.001075
S139	B	222	151862	177748	10207	632.7583	0.854367
S131	B	240	137512	139381	1014	572.9667	0.986591
S308	B	240	136998	186278	8181	570.825	0.735449
S330	B	240	136652	172980	9768	569.3833	0.789987
S356	C	240	130307	132612	1788	542.9458	0.982618
S268	C	240	129786	129370	736	540.775	1.003216

供货完成率用于刻画供应商对企业订单需求的实际兑现水平。数值越接近 1，代表供应商订单完成情况越好；小于 1 说明存在交付缺口，大于 1 则代表出现超量供货。

由供应商 S140 表现得数据可以发现，即使供应商的供货量总和较大，但供货完成率数值较低，仅有 0.629，说明该供应商虽然具备一定供给能力，但难以足额完成企业下达的订单，仅依靠供货规模会高估其生产保障价值。样本中存在部分供货完成率大于 1 的供应商，这类供应商可以稳定交付原材料，但超量供货可能造成企业库存压力。部分供应商供货完成率近似于 1，同时拥有较大的供货量总和。这类供应商的原材料供给充足，也可以较好兑现企业下达的订单要求，如样本中的 S275 和 S340，S340 供货量总和达到 171426，供货完成率为 0.996；S275 供货量总和为 158553，供货完成率为 1.002。二者在长期合作过程中，既能够输出大规模原材料，又可以将订单交付缺口控制在很小范围，对企业连续生产具备很高的保障价值，属于综合表现较为理想的合作供应商。

(2) 为衡量供应商供货的波动程度，引入有效供货周数据计算的供货量均方误差，以此反映供货稳定性。计算公式如下：

$$\varphi = \frac{1}{n} \sum_{i=1}^n (x_{i,t} - z_{i,t})^2 \quad (1)$$

其中 φ 为供货量均方误差， $z_{i,t}$ 为第 t 周供货商 i 的订货量。 n 为该供应商有效供货周数。该数值越小，代表供应商各周实际产出的供货量波动幅度微弱，每周输出水平相对固定，供货输出更加稳定，供货响应更有规律性；反之均方误差的数值越大，周度供货输出起伏剧烈，供货表现随机性较强。计算出的数值具体如下表所示（仅展示均方误差最大与最小的 10 家供应商，其余见附录）：

表 2 均方误差最小与与最大的各 10 家供应商样本

a 均方误差最小的 10 家供应商样本 b 均方误差最大的 10 家供应商样本

供应商 ID	材料分类	均方误差	供应商 ID	材料分类	均方误差
S397	B	0.033333	S108	B	802684.3
S033	C	0.033333	S139	B	934096.5
S351	C	0.0375	S160	C	1115597
S180	B	0.0375	S330	B	1189349
S206	C	0.041667	S308	B	1343484
S225	B	0.041667	S126	C	4430909
S044	C	0.045833	S151	C	5356326
S303	B	0.045833	S348	A	5727877
S026	A	0.045833	S140	B	13276696
S016	A	0.045833	S201	A	28412728

通过对比均方误差最小与最大的两组供应商样本，能够很直观地看到不同供应商在每周供货稳定性上的巨大的差异。均方误差数值最小的 10 家供应商，计算结果普遍维持在 0.1 以内。从原始数据不难看出，这类供应商只要进入供货周期，每周交付的原材料规模波动微小，供货输出保持在相对固定的区间。对企业来说，这类供应商的供给行为具备很强的可预见性，属于供货稳定性表现十分出色的合作对象。

再看均方误差最大的 10 家供应商，数值最大的已高达 28412728。均方误差属于绝对量指标，其中一部分供应商本身总供货体量就很高，较大的供货量会均方误差的计算结果升高，不可以理解为其完全不具备供货能力。即便存在这样的客观因素，巨大的数值依旧反映出，这些供应商周与周之间的产出差距十分悬殊，部分周次可以实现大规模供货，后续又会出现供不应求的现象。这种剧烈的供货波动，会影响企业的生产运营环节。到货规模不可预判，库存的预留空间就很难把控，既有可能遭遇原料不足，也会出现库存积压的现象，给库存调度、生产排产都增加不少现实压力。

（3）为进一步量化供应商的供给结构特征，本文继续引入供货规模占比与物料集中度两项指标，为后续建模部分提供更多维度的参考依据。

供货规模占比：该供应商供货量总和与全部供货商供货量总和之比，用来衡量某一家供应商，在全部供应商总供货量当中所占的份额。占比数值越高，代表这家供应商整体供货体量在所有合作对象里分量越重，企业原料供给对它的依赖程度也就越高。

物料集中度：企业需要 A、B、C 三种原材料，物料集中度刻画了供应商供货业务的集中程度，呈现该供应商的供货是分散在多种物料，还是集中在少数几类物料上。

用 python 计算出供货规模占比与物料集中度这两个指标的具体数值，结果如下表所示（因篇幅原因，仅展示 20 家数据，其余见附录）：

表 3 供应商供货规模占比与物料集中度描述性统计

供应 商 ID	材料 分类	供货规 模占比	物料集 中度	供应 商 ID	材料 分类	供货规 模占比	物料集 中度
S229	A	0.244235	0.006849	S131	B	0.091757	0.007463
S361	C	0.22653	0.008197	S308	B	0.091414	0.007463
S140	B	0.201544	0.007463	S330	B	0.091183	0.007463

S108	B	0.160777	0.007463	S356	C	0.089973	0.008197
S151	C	0.134295	0.008197	S268	C	0.089613	0.008197
S340	B	0.114386	0.007463	S306	C	0.087066	0.008197
S282	A	0.116541	0.006849	S194	C	0.06999	0.008197
S275	A	0.109117	0.006849	S348	A	0.063605	0.006849
S329	A	0.107716	0.006849	S352	A	0.061272	0.006849
S139	B	0.101332	0.007463	S143	A	0.056974	0.006849

从样本数据来看，供应商供货规模占比差异明显，最高 0.244、最低 0.057，供货规模占比供应商占据近四成份额，企业对其依赖程度更高，一旦供货波动会直接影响整体原料供给。

物料集中度普遍处于 0.0068-0.0082 的低位区间，但 C 类的物料集中度值相较于 A、B 类普遍偏高，该原材料每一家供应商的不可替代性最强；A 类原材料平均数值最小，供给风险小，企业的可选择性最大。

（4）在了解各供应商供给份额与物料结构之后，再从时间维度观察各类物料的到货特征，三类原材料周度供货量变化如图 1 所示

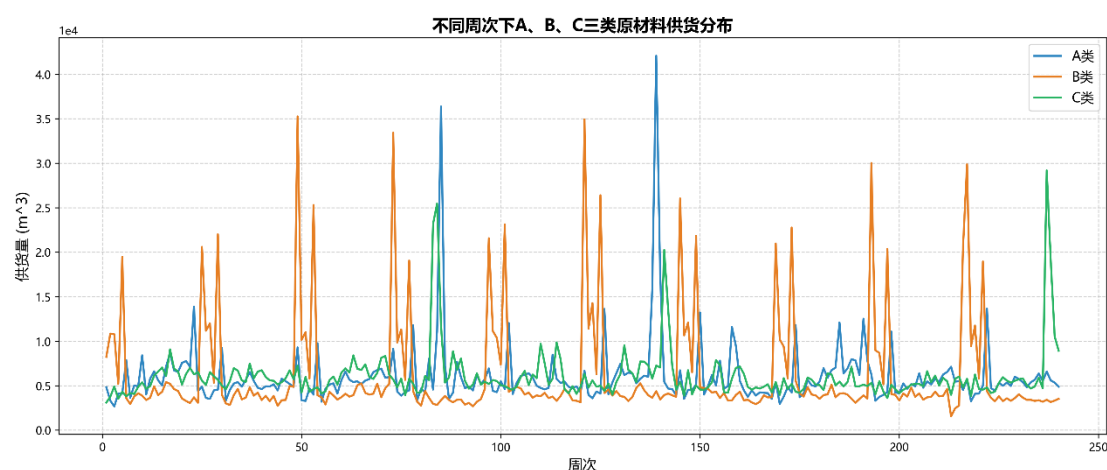


图 1 各周次下 A、B、C 三类原材料供货量变化趋势

从曲线变化能够看到，A、B、C 三类原材料的供货量分布均具有周期性，且供货规律周期为 24 周。三类原材料基线供货水平差距不大，且 B 类原材料会出现突发性大额供货周次，每周之间货量的起伏明显。

完成对供应商供给份额与物料结构指标的统计分析后，前文折线图从时间维度展示了原材料周度供货的波动特征。为进一步直观展现全部供应商之间的供货体量差异，绘制不同纵轴量程的散点图，结果如图 2 所示：

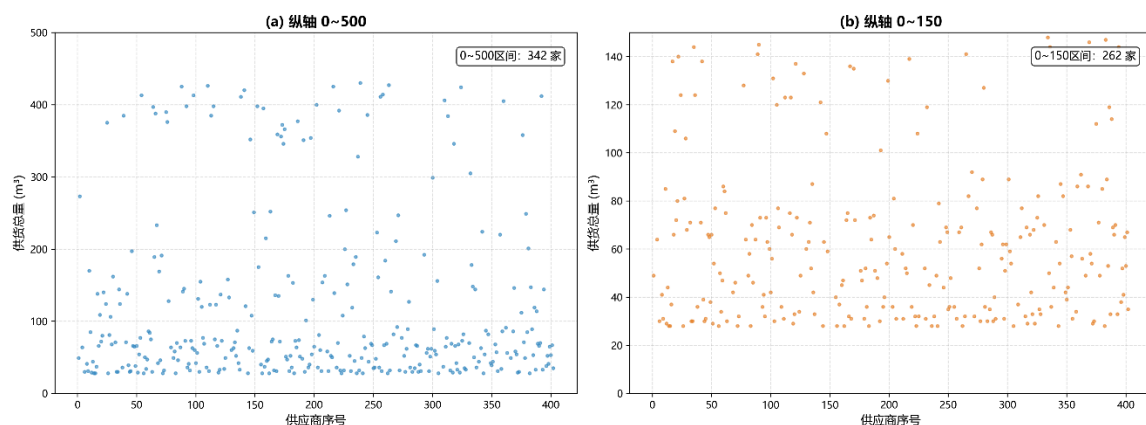


图 2 各供应商供货总量分布（不同纵轴量程对比）

从散点分布能够看出，供应商之间总供货量差距较大，少部分供应商承担了绝大部分供货任务。本文优先选取供货总量达到 80 的供应商作为后续分析对象。

5.1.2 基于熵权-TOPSIS 的供应商综合重要性评价

查阅相关文献^[2]可以发现，熵权-TOPSIS 组合模型常被用于多指标综合评价问题，熵权法的客观赋权特性可以降低主观因素干扰。因此本文采用该模型评价供应商重要性，从供货规模占比、供货完成率（履约率）、供货稳定性、物料可替代难度四个维度构建指标体系。以 402 家供应商为样本， $x_{i,j}$ 表示第 i 家供应商第 j 项指标。对各指标做极差归一化消除量纲影响，熵权法确定指标权重，再通过 TOPSIS 计算综合得分并降序排序，筛选前 50 家重要供应商。

（1）核心指标的筛选

为保证后续综合评价模型内部指标相互独立，避免两个指标之间相互影响，本研究针对供货规模占比、供货完成率（履约率）、供货稳定性、供应次数总和、物料集中度五项候选指标开展 Spearman 相关性分析^[3]。选取的这五项指标，是希望从不同角度刻画供应商的综合价值：供货规模占比体现体量贡献，供货完成率反映订单交付完成水平，供货稳定性用来衡量供货波动可控程度，供应次数总和可以侧面体现双方合作牢固程度，物料集中度则表征该供应商的不可替代性，几个指标之间可以形成较好的维度互补，分析结果见图 3。

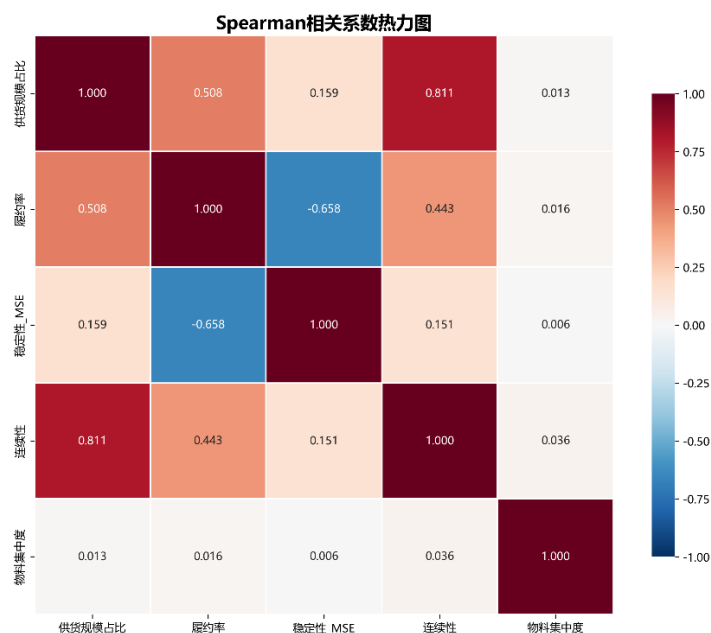


图 3 候选评价指标 Spearman 相关系数热力图

注：图中的履约率就是本文的供货完成率；连续率就是本文的供应次数总和。

由上图可知，供货规模占比与供应次数总和的相关系数高达 0.811，大于 0.7 的共线性判别阈值，两项指标存在高度正相关性。对比两项指标的实际业务含义，供货规模占比能够直接体现供应商的供货份额，对生产保障能力的刻画会比供应次数总和更加直观。因此研究剔除供应次数总和这一指标。经过相关性筛选之后，最终保留四项评价指标：**供货规模占比、供货完成率、供货稳定性、物料集中度**。

(2) 构建原始决策矩阵

提取 402 家供应商对应的四项评价指标原始观测值，构建评价模型的原始决策矩阵。矩阵的每一行对应一家供应商，每一列分别对应供货规模占比、供货完成率、供货稳定性、物料集中度四项筛选后的评价指标。

记 $x_{i,j}$ 为原始决策矩阵内元素，代表第 i 个供应商的第 j 项指标的原始取值。

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ \vdots & \vdots & \vdots & \vdots \\ x_{402,1} & x_{402,2} & x_{402,3} & x_{402,4} \end{bmatrix} \quad (2)$$

不同指标之间量纲、数值量级差异明显，供货规模占比为比例数值，供货稳定性代表供货量的误差平方，物料集中度由供应商数量倒数计算得到，各指标无法直接开展后续熵权运算，因此需要对原始矩阵做极差归一化处理。

(3) 极差归一化

本文中供货稳定性为误差平方量级，高达百万，若直接用原始数据计算，该指标将在欧式距离中主导结果，为消除不同的量纲影响，采用极差归一化使所有指标都在同一个尺度[0,1]上，再进行比较。

正向指标（越大越好）：

$$z_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)} \quad (3)$$

逆向指标（越小越好）：

$$z_{ij} = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)} \quad (4)$$

归一化后得到的标准化矩阵：

$$Z = \begin{bmatrix} z_{11} & z_{12} & z_{13} & z_{14} \\ z_{21} & z_{22} & z_{23} & z_{24} \\ \vdots & \vdots & \vdots & \vdots \\ z_{402,1} & z_{402,2} & z_{402,3} & z_{402,4} \end{bmatrix} \quad (5)$$

(4) 熵权法确定权重

经过极差归一化处理后得到标准化矩阵 Z ，矩阵内所有指标取值均处于[0,1]区间。为了避免主观赋权带来的人为偏差，本文采用熵权法依据数据自身的离散程度计算各指标客观权重。

共有 $n=402$ 个供应商样本， $m=4$ 项指标。

Step1: 计算第 j 个指标下第 i 个供应商的占比：

$$p_{ij} = \frac{z_{ij}}{\sum_{i=1}^{402} z_{ij}} \quad (6)$$

Step2:计算第 j 个指标的熵值:

$$e_j = -\frac{1}{\ln 402} \sum_{i=1}^{402} p_{ij} \ln(p_{ij}) \quad (7)$$

Step3:计算第 j 个指标的权重:

$$w_j = \frac{1 - e_j}{\sum_{j=1}^4 1 - e_j} \quad (8)$$

通过上述公式计算，得到四项指标权重分别为供货规模占比 $w_1=0.7301$ ，供货完成率 $w_2=0.1104$ ，供货稳定性 $w_3=0.0009$ ，物料集中度 $w_4=0.1586$ 。将权重结果绘制成柱状图，可以把各指标权重大小可视化，如下图所示：

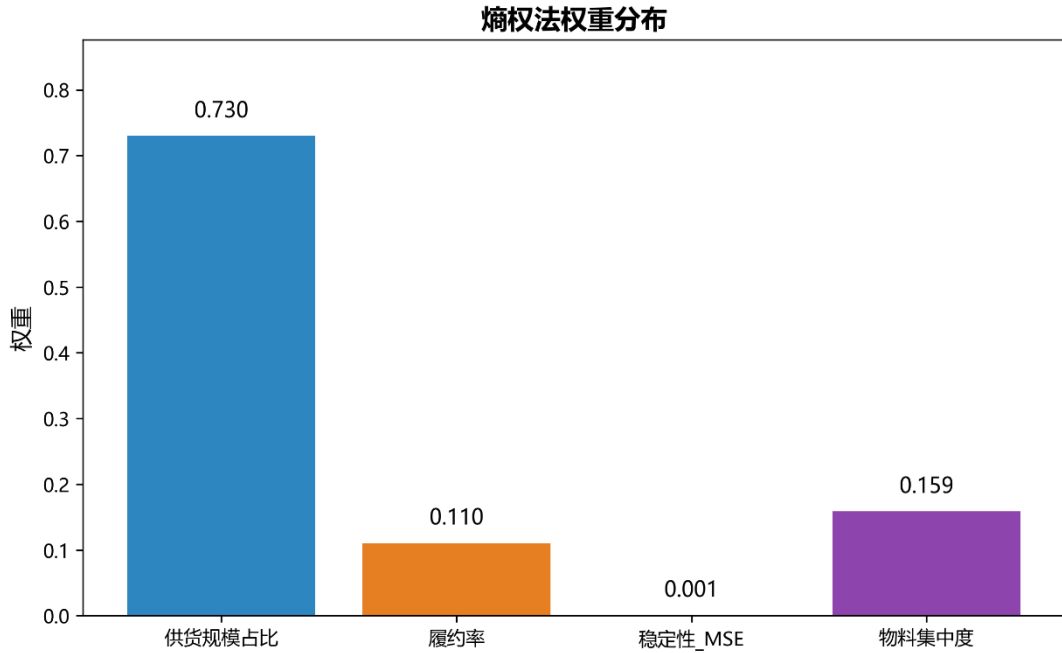


图 4 熵权法各指标权重分布

注：图中履约率是本文中的供货完成率；图中权重是近似后的数值。

从图中不难看出，供货规模占比权重达到 0.730，占据主导地位，说明在本次样本中，供应商供货份额是区分供应商重要程度的核心数据；供货稳定性权重仅为 0.001，几乎不起作用。该现象来源于样本数据本身，大部分供应商的周度供货波动数值比较集中，数据离散程度很低，能够提供的有效区分信息有限。虽然该指标业务层面可以反映供货稳定性，但受样本分布限制，所以得到的权重极低。本文依旧保留该指标参与后续 TOPSIS 运算，保证评价维度的完整性。

(5) TOPSIS 综合评价

使用熵权法已经得到四项指标对应的客观权重，为进一步量化每一家供应商的综合重要水平，本文采用 TOPSIS 法开展综合评价。该方法结合标准化矩阵与指标权重构建加权决策矩阵，通过计算样本与正、负理想解之间的距离，得到各供应商的综合贴近度，以此完成多对象的优劣排序。

Step1: 构造加权归一化矩阵:

$$v_{ij} = w_j z_{ij} \quad (9)$$

Step2:确定正理想解和负理想解:

$$V^+ = (v_1^+, v_2^+, v_3^+, v_4^+) = \left(\max_i v_{i1}, \max_i v_{i2}, \max_i v_{i3}, \max_i v_{i4} \right) \quad (10)$$

$$V^- = (v_1^-, v_2^-, v_3^-, v_4^-) = \left(\min_i v_{i1}, \min_i v_{i2}, \min_i v_{i3}, \min_i v_{i4} \right) \quad (11)$$

由于归一化后 $\max(z_{ij})=1, \min(z_{ij})=0$, 所以:

$$V^+ = (\omega_1, \omega_2, \omega_3, \omega_4), \quad V^- = (0, 0, 0, 0) \quad (12)$$

Step3:计算正负理想解的欧氏距离:

$$D_i^+ = \sqrt{\sum_{j=1}^4 (v_{ij} - \omega_j)^2} \quad (13)$$

$$D_i^- = \sqrt{\sum_{j=1}^4 (v_{ij} - 0)^2} \quad (14)$$

Step4: 计算相对贴进度 (TOPSIS 得分):

$$C_i = \frac{D_i^-}{D_i^- + D_i^+} \quad (15)$$

$C_i \in [0, 1]$, 值越大表示供应商越接近最优状态, 越重要。

上述全部计算公式通过 Python 编程实现, 完成加权矩阵构造、正负理想解的求解、欧氏距离计算以及综合贴进度 C_i 批量运算。最终得到 402 家供应商各自的综合评价得分。

(6) 排序与决策

将全部 402 家供应商依据 C_i 的大小做降序排序, 综合得分靠前的供应商, 在供货体量、供货完成率、物料集中度等多方面整体表现更突出。本文选取排序结果里排名前 50 的供应商作为关键供应商, 后续订货优化模型将主要围绕这 50 家主体开展分析。详细数据如下表所示:

表 4 50 家最重要的供货商

供应商 ID	材料分类	得分	供应商 ID	材料分类	得分
S361	C	0.91943	S126	C	0.228487
S229	A	0.819666	S307	A	0.225185
S140	B	0.786475	S201	A	0.223198
S108	B	0.647978	S395	A	0.217755
S151	C	0.566992	S294	C	0.209547
S340	B	0.473447	S080	C	0.209406
S282	A	0.463246	S076	C	0.209278
S275	A	0.435853	S206	C	0.208545
S329	A	0.430654	S213	C	0.207384
S139	B	0.421429	S218	C	0.207273

S356	C	0.410703	S067	C	0.206017
S268	C	0.409836	S342	C	0.205726
S306	C	0.401134	S244	C	0.205167
S131	B	0.387828	S253	C	0.204968
S330	B	0.381834	S113	C	0.204482
S308	B	0.381708	S149	C	0.203911
S194	C	0.344861	S025	C	0.203566
S352	A	0.259673	S133	C	0.201598
S247	C	0.257956	S003	C	0.201125
S348	A	0.256279	S379	C	0.200706
S374	C	0.246035	S362	C	0.200652
S284	C	0.242421	S351	C	0.200298
S037	C	0.241109	S305	C	0.199998
S143	A	0.240184	S044	C	0.199684
S365	C	0.23507	S135	C	0.198412

5.2 问题二

5.2.1 第一小问

(1)数据预处理

附件 1 中的原始供货数据存在大量零记录,这些零值一部分来源于企业未向该供应商下达订单指令,另一部分来源于供应商实际供货不足。若直接使用全部 240 周的简单均值,会被企业未订货而产生的大量零值拉低供应商真实的供应能力,造成结果低估。因此本文采用有订单平均供货量对供应商的实际供货水平进行刻画,仅保留供应商存在实际供货记录的周次进行统计与计算:

$$\mu_i = \frac{1}{N_i} \sum_{t: s_{i,t}} s_{i,t}, \quad N_i = \sum_{t=1}^{240} 1_{\{s_{i,t} > 0\}} \quad (16)$$

式中 $1_{\{s_{i,t} > 0\}}$ 为指示函数,条件成立取值为 1, 否则为 0; N_i 代表供应商 i 产生实际供货记录的周数。这么处理,可以直接把企业决策层面带来的无效 0 样本直接过滤掉,还原出供应商本身的供货水准。

本文中 A、B、C 三类原材料对应不同的成品消耗系数,意思是生成相同的产品所消耗 A、B、C 不同,因此原材料供货量不可以直接相加对比。为统一量纲,将各供应商的原材料供应能力折算为等效成品产能,把原料供应能力转化为可生产出的成品体积:

$$E_i = \frac{\mu_i}{r_m(i)} \quad (17)$$

各类原材料对应的其中消耗分别为 $r_A=0.6$, $r_B=0.66$, $r_C=0.72$ 。 E_i 表征企业向供应商正常下达订单时,供应商提供的平均供货原材料能够转化得到的成品规模,实现不同原料类别供应商供应能力的横向对比与累加。

(2) 目标函数的建立

得到等效成品产能后,构建 0-1 整数规划模型。在保障生产的前提下,使选

取的供应商总数量最小化。

$$\min N = \sum_{i=1}^{402} y_i \quad (18)$$

模型设置两条硬性约束约束条件：

1) 产能覆盖约束：选中的供应商等效成品产能总和需要满足企业每周 28200m³ 的成品生产压力。

$$\sum_{i=1}^{402} E_i y_i \geq P = 28200 \quad (19)$$

2) 0-1 决策变量约束：

$$y_i \in \{0,1\}, i = 1,2, \dots, 402 \quad (20)$$

模型以全部 402 家供应商为候选池，决策变量 y_i 对全体供应商生效，做出这样的设置时考虑到第一问筛选的 50 家供应商是“综合最优”的，但不预先假设其产能一定充足，因此允许模型在必要时向 50 名之后的供应商扩展。

当然，候选池覆盖全部供应商，并不意味着模型会无规则选取综合评价靠后的主体。供应商参与产能累加的先后顺序严格遵循问题一得到的综合得分降序排列，综合得分更高的供应商优先参与产能汇总。因次原先筛选出的 50 家优质供应商会处于累加序列的靠前位置，具备优先被选用的地位。

(3)模型的求解

针对这套模型，本文选择贪心累加的方式完成求解。把全体供应商按照综合得分从高到低排好顺序，顺着排名依次累加各家的等效成品产能 E_i 。累加过程持续观察累计产能，一旦总和跨过每周产能阈值，立刻停止计算，此刻统计到的供应商数量，就是我们想要的最低数目。计算结果如下表所示：

表 5 贪心累加计算结果

供应商 ID	材料分类	累计 E_i	供应商 ID	材料分类	累计 E_i
S361	C	1898.611	S330	B	18547.92
S229	A	4363.104	S308	B	19412.8
S140	B	6452.816	S194	C	19999.4
S108	B	7973.965	S352	A	20617.68
S151	C	9099.532	S247	C	20945.79
S340	B	10181.77	S348	A	21739.78
S282	A	11357.74	S374	C	22024.64
S275	A	12458.8	S284	C	22294.3
S329	A	13545.73	S037	C	22742.69
S139	B	14582.19	S143	A	23317.6
S356	C	15336.28	S365	C	23558.52
S268	C	16087.36	S126	C	24284.1
S306	C	16817.08	S307	A	25046.25
S131	B	17685.21	S201	A	29926.54

由计算结果可看出，累加到第 27 家供应商时，累计等效产能为 25046.25，不满足产能需求，而累加到第 28 家供应商时，累计等效产能为 29926.54，满足

企业产能需求，且超出 6.12%。也就是说，想要稳住企业每周的产能至少需要合作 28 家供应商。

表 6 选中的 28 家供应商原材料类别分布

原料类别	供应商数量	占比
A	9	32.14%
B	7	25.00%
C	12	42.86%
总计	28	100%

最终选出的 28 家供应商完整覆盖 A、B、C 三类原材料，其中 C 类供应商数量最多，B 类数量最少。该结果是贪心算法按综合得分降序累加带来的自然结果，反映出在综合得分靠前的供应商群体当中，C 类原料的优质供给主体数量较多。

5.2.2 第二小问：建立混合整数线性规划模型

(1) 每周原料消耗量的计算

在开展模型构建之前，先对每周原料消耗量这一固定外生参数进行预先计算。由于 α_m 为固定参数，每周各类原料的消耗量由下式计算：

$$D_m = P\alpha_m r_m, m \in \{A, B, C\} \quad (21)$$

每周三类原料总消耗量为各类原料消耗量求和：

$$D = \sum_{m \in \{A, B, C\}} D_m = P \sum_{m \in \{A, B, C\}} \alpha_m r_m \quad (22)$$

进一步统计 402 家供货商 24 周全部采购量的原料构成，各类原材料采购占比结果如图 5 所示：

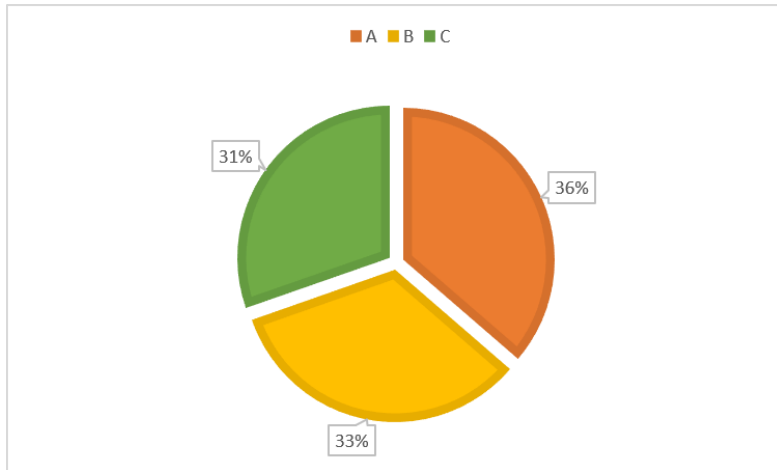


图 5 24 周周期内 A、B、C 三类原材料总采购量占比

结合上图可以看出，24 周周期内三类原料的采购规模整体较为接近。A 类原材料采购总量 146，占全部采购量的 36%，在三类原料中采购规模最高；B 类原材料采购总量 134，占比 33%；C 类原材料采购总量 122，占比 31%，为三者当中最低。

根据企业历史生产数据，三类原料用量比例近似相等，取 $\alpha_A = \alpha_B = \alpha_C = 1/3$ ，代入后得到每周原料总消耗：

$$D = 28200 \times \frac{1}{3} \times (0.6 + 0.66 + 0.72) = 28200 \times 0.66 = 18612 \quad (23)$$

D 的单位为立方米\周。

(2) 决策变量的定义

$X_{i,t}$: 第 t 周向供应商 i 的原料订货量;

$U_{i,k,t}$: 0-1 变量, 第 t 周是否将供应商 i 的货物交由转运商 k 运输;

I_t : 第 t 周期结束后的期末原料库存量。

(3) 目标函数的建立

模型的目标函数综合考量原材料采购成本、库存持有成本以及转运环节带来的货物损耗成本:

$$\min Z_2 = \sum_{i \in J^*} \sum_{t=1}^{24} c_i \cdot x_{i,t} + \sum_{t=1}^{24} h I_t + \sum_{i \in J^*} \sum_{k=1}^8 \sum_{t=1}^{24} c_i \frac{\lambda_{k,t}}{100} \cdot x_{i,t} \cdot u_{i,k,t} \quad (24)$$

其中 J^* 为第一小问确定的 28 家供应商集合。

(4) 约束条件

1). 引入库存动态平衡方程刻画每周库存流转关系, 周期期末库存由上一周期库存、扣除转运损耗后的实际到货量与当期生产原料消耗量共同决定:

$$I_t = I_{t-1} + \sum_{i \in J^*} \sum_{k=1}^8 \left(1 - \frac{\vartheta_{k,t}}{100}\right) \cdot x_{i,t} \cdot u_{i,k,t} - D, \forall t \quad (25)$$

2). 为抵御供应商供货波动带来的断供风险, 设置动态安全库存约束, 各周期期末库存不得低于两周生产所需原料总量; 同时增加库存上限, 避免原料过度积压占用企业流动资金:

$$\begin{cases} I_t \geq 2 \times D, \forall t = 1, 2, \dots, 24 \\ I_t \leq 8 \times D, \forall t \end{cases} \quad (26)$$

3). 考虑供应商实际供给能力, 企业每周向供应商下达的订货量不能超过该供应商历史最大供货规模:

$$0 \leq x_{i,t} \leq \max_{t' \in \{1, \dots, 240\}} S_{i,t'}, \forall i, \forall t \quad (27)$$

4). 转运业务层面, 规定每家供应商每周的货物必须指派给且仅可指派给一名转运商; 同时对转运商单周承接货物总量加以限制, 不得超出其运力上限:

$$\begin{cases} \sum_{k=1}^8 u_{i,k,t} = 1, \forall i \in J^*, \forall t \\ \sum_{i \in J^*} x_{i,t} \cdot u_{i,k,t} \leq 6000, \forall k, \forall t \end{cases} \quad (28)$$

5). 最后补充变量取值约束, 订货量与库存为非负连续变量, 转运指派变量取 0-1 整数:

$$x_{i,t} \geq 0, I_t \geq 0, u_{i,k,t} \in \{0, 1\} \quad (29)$$

该模型为混合整数线性规划(MILP), 满足线性求解条件, 使用 python 完成计算, 得到 24 周的订货量、转运指派以及期末库存的全套最优方案。完整周度的明细数据置于附件 A。

5.2.3 第三小问

求解该混合整数线性规划模型后,可输出 24 周规划周期内各供应商订货量、转运商指派关系以及每周期末库存的完整决策结果。下文将从库存运行状态、成本开销、供应商订货结构三个维度,对求解得到的订货-转运方案开展效果分析。

(1) 库存运行状态分析

安全库存是保障企业生产不间断的硬性条件,因此首先校验全周期库存是否满足约束要求。提取 24 周每期期末库存 I_t , 与两周安全库存阈值 $2D=37224.00$ 立方米做对比。

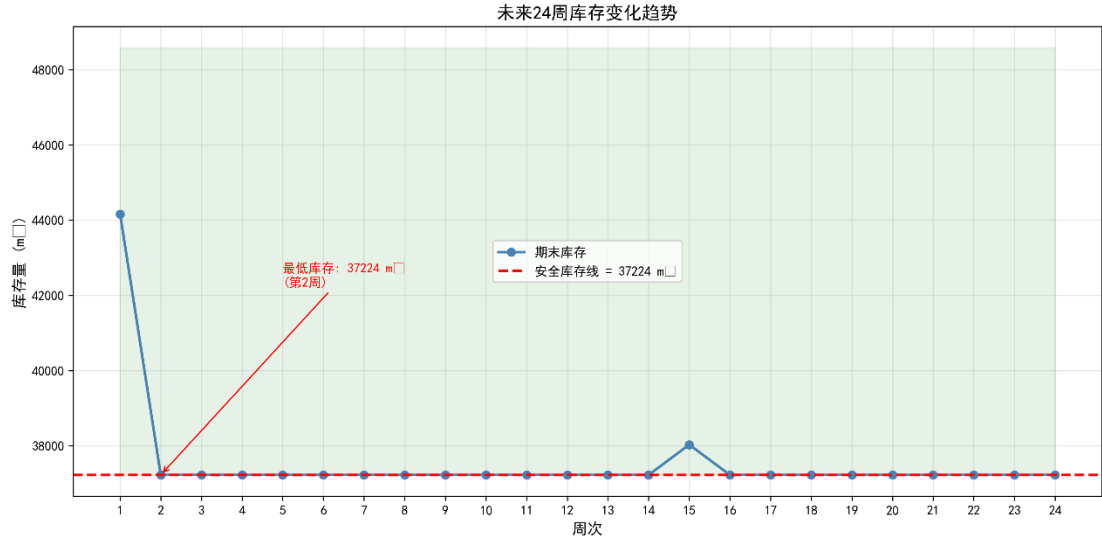


图 6 24 周周期库存波动与安全库存对比趋势图

上图反映了规划周期内库存的动态变化。整个周期中,第 2 周期末库存降至 37224.00 立方米,刚好触及安全库存下限,安全缓冲空间基本耗尽;第 1 周库存水平最高,达到 44164.18 立方米;24 周平均库存维持在 37546.70 立方米。纵观全部周次,期末库存均没有跌破安全库存底线,从库存层面证明这套方案可以规避原料短缺风险,维持生产持续开展。

(2) 成本开销分析

确认方案可行之后,进一步考察方案的经济表现,各项成本统计结果如下表所示:

表 7 成本构成及占比统计表

成本项目	金额 (元)	占比
采购成本	446, 118. 57	90. 31%
库存成本	45, 056. 04	9. 12%
损耗成本	2, 824. 48	0. 57%
总成本	493, 999. 08	100. 00%

模型输出总方案成本为 493999.08 元。其中原材料采购成本高到 446118.57 元,占到总成本的 90.31%,是成本支出的主要部分。库存持有成本占比 9.12%,说明模型没有一味大量的囤货,在守住安全库存底线的前提下尽量压低原料积压带来的持有开销。运输损耗成本仅占总成本 0.57%,这也侧面反映出,转运商在分配环节,模型会倾向选择损耗表现更优的转运商,把运输环节带来的额外损失

控制在较低水平。图 7 直观呈现各项成本的占比分布。

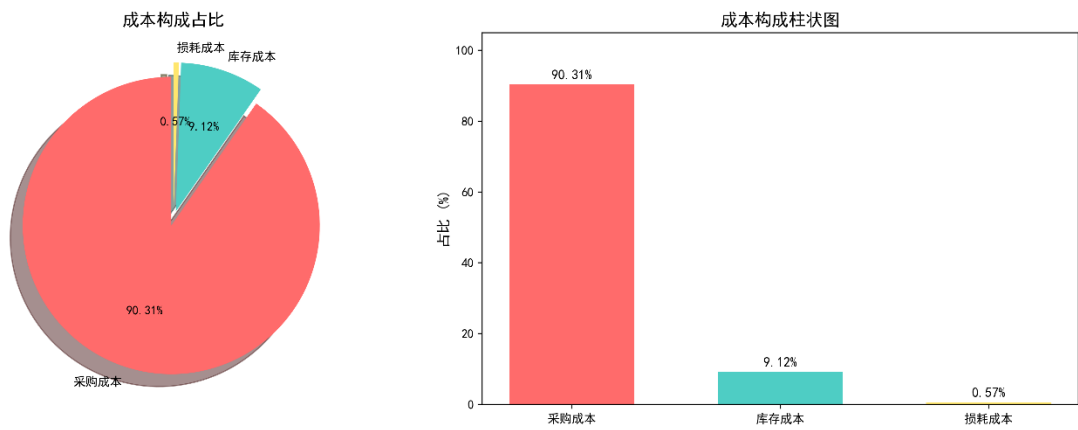


图 7 方案各项成本占比可视化

(3) 供应商订货结构分析

仅满足约束和成本最优还不够，还需要结合供应商订货分布，判断方案在实际业务场景下是否具备落地条件。统计订货规模靠前的供应商，前 10 家供应商订货数据如下表所示：

表 8 前 10 大供应商采购量统计

排名	供应商编号	材料类型	采购量 (m³)
1	S361	C	62,882.00
2	S140	B	51,062.91
3	S151	C	38,418.40
4	S268	C	25,597.80
5	S139	B	25,400.14
6	S306	C	25,219.20
7	S108	B	25,122.21
8	S356	C	23,613.67
9	S126	C	23,192.47
10	S340	B	21,038.27

从求解出来的数据可以看出，S361 的总订货量遥遥领先，是最重要的供货来源；S140 订货量次之；排名 3-10 位供应商订货量逐步下降，但仍承担一定供货任务。前 10 家供应商内部订货规模差距明显，头部少数供应商承接了主要供货任务。

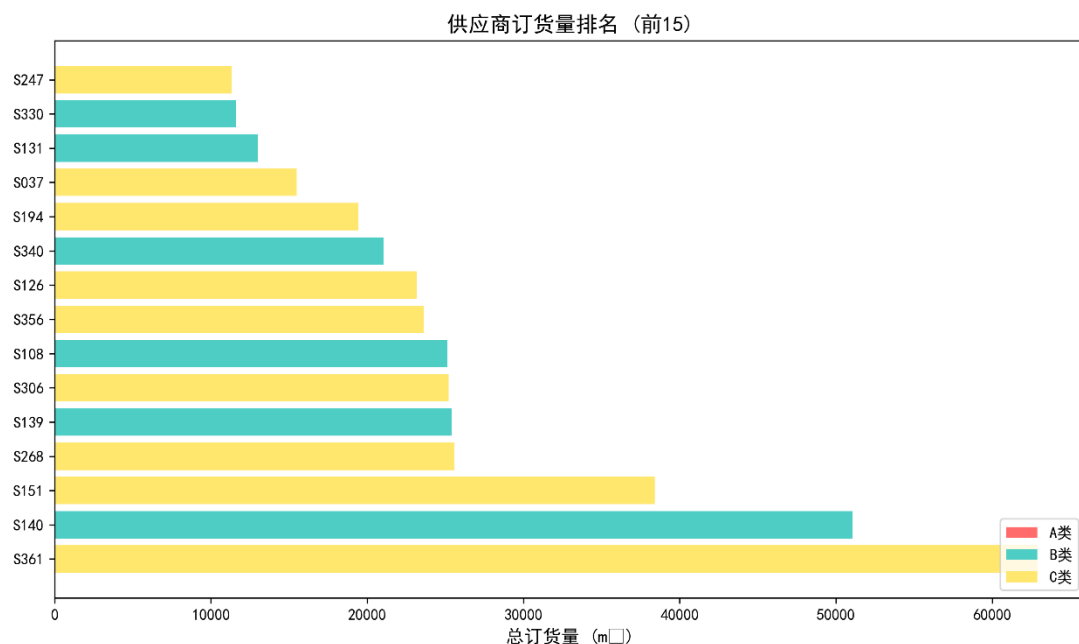


图 8 前 15 位供应商订货量对比

上图条形图形式直观呈现前 15 家供应商订货规模对比。图形可以更加直观地体现订货量的衰减特征：前两家供应商条形高度显著高于其余供应商，往后条形高度逐步降低，尾部多家供应商订货量处于较低水平。该分布说明模型订货呈现明显的头部集中特征。这种适度集中的订货模式，便于企业开展订单对接与沟通工作。

再统计全部周期内不同类别原料的总采购规模，结果如下表所示：

表 9 各类原材料采购量及占比

材料类型	采购量 (m³)	占比
A	0.00	0.00%
B	158,303.84	36.79%
C	271,984.34	63.21%
合计	430,288.18	100.00%

计算结果显示 A 类原料总采购量为 0，货源主要由 B、C 两类原料支撑。出现该现象并不是模型出错，而是不同原料采购价格存在差异；模型以总成本最小为优化目标，在满足生产约束的前提下，会主动更多选用采购成本更低的 B、C 类原料，减少高价 A 类原料的采购。

(4) 实施效果综合评价

综合库存状态、成本开销、订货结构三方面结果来看，该套订货-转运方案在 24 周规划周期内严格遵守安全库存约束，能够有效规避生产断供风险。整体成本水平可控，运输损耗造成的额外损失被抑制在较小范围。订货相对集中的分配模式也便于企业实际执行。整体而言，该混合整数线性规划模型输出的方案兼顾可行性与经济性，可以为企业未来 24 周的原料采购以及转运调度提供决策参考。

5.3 问题三

5.3.1 建模与求解过程

(1) 场景的设定思路

由问题二求解结果可知，充足的 A 类供应商资源，却没有拿到任何采购份额，因为 A 类原材料最贵，而问题二目标函数为总成本最小化，A 类原材料最贵。想要完成原料结构的改造，不能只靠单一方案做判断。本文构造多组带有不同偏好强度的约束场景，为后续熵权法客观赋权提供样本。

选取 15% 定为 A 类采购占比的初始下限。选择这个数值，有两方面现实原因。该水平约为全部样本里 A 类供应商占比的四成，可以提高 A 类采购的优化导向，也不会因为目标过高导致模型无解。后续以 3% 作为步长向上抬升约束，依次得到 15%、18%、21%、24% 四组约束条件，再叠加一组无约束的基准场景，一共得到 5 组可供对比的方案样本，满足熵权法的计算要求。

(2) 场景参数的设置

表 10 多场景参数的设置情况

场景编号	A 类占比下限 θ_A	C 类占比上限 θ_C	偏好强度
场景 0 (基准)	不设约束	不设约束	无偏好
场景 1	$\geq 15\%$	$\leq 48\%$	温和偏好
场景 2	$\geq 18\%$	$\leq 45\%$	中等偏好
场景 3	$\geq 21\%$	$\leq 42\%$	较强偏好
场景 4	$\geq 24\%$	$\leq 39\%$	激进偏好

A 类占比下限与 C 类占比上限联动调整，A 类原材料占比每提升 3%，C 类原材料占比上限相应降低 3%， $\theta_A = \theta_C = 63\%$ 始终保持恒定，与问题二中 A 和 C 总站比一致，避免 C 类因约束过紧导致无可行解。

(3) 带约束的 MILP 模型构建

模型主体沿用问题二搭建完成的混合整数线性规划框架，只额外新增 A、C 两类原材料采购占比的约束式子。库存动态平衡、安全库存阈值、供应商供货上限、转运商运力天花板等其余约束条件，全部继承原有设定。

A 类原材料采购占比约束：

$$\frac{\sum_{i \in J^*} \sum_{t=1}^{24} x_{i,t}}{\sum_{i=1}^{28} \sum_{t=1}^{24} x_{i,t}} \geq \theta_A \quad (30)$$

C 类原材料采购占比约束：

$$\frac{\sum_{i \in J^*} \sum_{t=1}^{24} x_{i,t}}{\sum_{i=1}^{28} \sum_{t=1}^{24} x_{i,t}} \leq \theta_C \quad (31)$$

分别对 5 个场景独立求解，记录每一组方案对应的总成本 f_1 、A 类采购占比 f_2 、C 类采购占比 f_3 以及转运总损耗率 f_4 ，汇总得到各场景的目标结果，如表 11 所示：

表 11 不同约束场景下方案目标结果

方案	总成本 f_1	A 类占比 f_2	C 类占比 f_3	总损耗率 f_4
场景 0 (基准)	493999.08	0.00%	63.00%	0.57%
场景 1 (15%)	501523.29	15.00%	48.00%	0.08%

场景 2 (18%)	504123.82	18.00%	45.00%	0.08%
场景 3 (21%)	506657.65	21.00%	42.00%	0.08%
场景 4 (24%)	509155.53	24.00%	39.00%	0.07%

从表中能够观察到几组很直观的变化规律。随着 A 类采购占比约束不断收紧, 模型被迫接纳更多 A 类供应商订单, 整体总成本随之逐步抬升。每抬高 3 个百分点的 A 类采购占比, 总成本大致上涨 2500 元左右, 成本增幅不算剧烈, 说明在现有供应商池内提升 A 类采购具备现实可行性。C 类采购占比跟随约束同步下降, 完全符合场景设置的预期。

(4) 熵权法的权重计算与修正

在不同备选方案之间结果波动幅度很大, 就代表这项指标携带的区分信息更多, 理应分配更高的权重; 反过来, 指标在各个方案中输出结果差别不大, 能提供的有效信息有限, 对应的权重就应当压低。

具体计算步骤如下:

1) 指标标准化处理

A 类采购占比属于正向指标, 数值越大代表效果越好:

$$y_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)} \quad (32)$$

总成本、C 类采购占比、转运损耗率属于负向指标, 数值越小越符合预期:

$$y_{ij} = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)} \quad (33)$$

2) 计算各方案特征比重

$$p_{ij} = \frac{y_{ij}}{\sum_{i=1}^5 y_{ij}} \quad (34)$$

3) 计算指标熵值

$$e_j = -\frac{1}{\ln(5)} \sum_{i=1}^5 p_{ij} \ln(p_{ij}) \quad (35)$$

4) 计算初始熵权

$$w_j = \frac{1 - e_j}{\sum_{k=1}^4 (1 - e_k)} \quad (36)$$

将表 13 中场景 1-4 的 4 组核心样本数据代入上述计算流程, 得到各指标的熵值以及未经调整的原始熵权, 结果由表 12 所示:

表 12 熵权法原始计算结果

评价指标	指标方向	原始熵值 e_j	原始差异系数 d_j	原始熵权
总成本 f_1	越小越好	0.727606	0.272394	15.02%
A 类采购占比 f_2	越大越好	0.729574	0.270426	14.91%
C 类采购占比 f_3	越小越好	0.729574	0.270426	14.91%
转运损耗率 f_4	越小越好	≈ 0	1.000000	55.15%

熵权法计算结果显示损耗率权重高达 55.15%，但观察表中五个场景的损耗率数据可以发现：除基准场景（0.57%）外，场景 1 至场景 4 的损耗率均在 0.07%~0.08%之间，几乎无差异。熵权法将损耗率权重放大至 55%的原因在于：损耗率的极差（0.57%-0.07%=0.50%）主要由基准场景的偏高值贡献，在熵权法的标准化处理中，这一异常值被转化为接近 1 的标准化值，而其余四个场景的标准化值接近 0，由此放大了损耗率的数据差异，从而计算出高权重。但在问题三的核心对比中，我们关注的是 15%~24%四个偏好场景之间的方案选择，损耗率在这些场景中无法有效区分方案优劣。若直接采用 55%的权重，将导致优化结果主要由损耗率驱动，与企业实际决策中“成本与 A/C 结构调整为核心目标”的认知不符。

因此，本文在熵权法计算结果基础上进行修正^[4]：将损耗率权重固定为 10%，剩余 90%按其他三个目标的原始比例分配，具体计算公式如下：

$$\begin{cases} w'_1 = 0.90 \times \frac{w_1}{w_1 + w_2 + w_3} \\ w'_2 = 0.90 \times \frac{w_2}{w_1 + w_2 + w_3} \\ w'_3 = 0.90 \times \frac{w_3}{w_1 + w_2 + w_3} \\ w'_4 = 0.10 \end{cases} \quad (37)$$

经过换算得到修正后的权重集合，结果如下表所示：

表 13 修正后的各项目目标权重

评价指标	修正后权重
总成本 f1'	30.15%
A 类采购占比 f2'	29.93%
C 类采购占比 f3'	29.93%
转运损耗率 f4'	10.00%

这样处理，相当于保留熵权法对总成本、A 类、C 类三项指标的数据驱动特性，三者之间权重相对大小完全由样本数据决定；同时削弱损耗率这项在高 A 占比区间区分度不足指标的话语权，实现客观计算结果和现实决策经验之间的折中。

（5）加权多目标 MILP 模型构建

本文在场景 0 基准方案下的各项目目标数值做归一化处理，消除量纲带来的干扰，结合修正完成的权重，把多目标优化问题转化为单目标求解。需要注意 A 类占比是最大化目标，所以在目标函数表达式中该项取负号：

$$\min Z = w'_1 \frac{f_1}{f_1^{\text{基准}}} - w'_2 \frac{f_2}{f_2^{\text{基准}}} + w'_3 \frac{f_3}{f_3^{\text{基准}}} + w'_4 \frac{f_4}{f_4^{\text{基准}}} \quad (38)$$

其中 $f_j^{\text{基准}}$ 代表场景 0 基准方案对应的各项目目标数值。

原料结构调整的要求，全部融入到目标函数当中。其余约束条件全部继承问题二的模型设定。

1) 库存动态平衡约束

$$I_t = I_{t-1} + \sum_{i \in \mathcal{I}^*} \sum_{k=1}^8 \left(1 - \frac{\vartheta_{k,t}}{100}\right) \cdot x_{i,t} \cdot u_{i,k,t} - D, \forall t \quad (39)$$

2) 安全库存约束, 保证期末库存至少能够支撑两周生产物料消耗

$$I_t \geq 2D, \forall t \quad (40)$$

3) 供应商供货能力上限约束

$$0 \leq x_{i,t} \leq \max_{t'}(S_{i,t'}), \forall i, t \quad (41)$$

4) 转运商指派唯一性约束, 同一供应商同一周的订货只能分配给一位转运商

$$\sum_{k=1}^8 u_{i,k,t} = 1, \forall i, t \quad (42)$$

5) 转运商每周运力上限约束

$$\sum_i x_{i,t} u_{i,k,t} \leq 6000, \forall k, t \quad (43)$$

6) 变量非负性与整数性约束

$$x_{i,t} \geq 0, I_t \geq 0, u_{i,k,t} \in \{0, 1\} \quad (44)$$

7) 为防止模型在“A类占比最大化”的目标驱动下将A类比例推至极端值(如100%), 导致方案失去实际可行性, 本文在场景设计的24%下限和39%上限基础上, 增设A类占比上限32%和C类占比下限28%, 构成完整的可行区间:

$$\begin{cases} 24\% \leq \frac{\sum_{i \in \mathcal{I}_A} \sum_{t=1}^{24} x_{i,t}}{\sum_{i=1}^{28} \sum_{t=1}^{24} x_{i,t}} \leq 32\% \\ 28\% \leq \frac{\sum_{i \in \mathcal{I}_C} \sum_{t=1}^{24} x_{i,t}}{\sum_{i=1}^{28} \sum_{t=1}^{24} x_{i,t}} \leq 39\% \end{cases} \quad (45)$$

其中 $\mathcal{I}_A$ 和 $\mathcal{I}_C$ 分别为A类和C类供应商集合。

该优化模型属于混合整数线性规划(MILP), 用python求解出24周周期内各供应商最优订货量 $x_{i,t}^*$ 、转运商具体指派方案 $u_{i,k,t}^*$ 以及每一周的期末库存 I_t^* , 得到一套兼顾成本开销、原料品类结构、转运损耗水平的综合订购-转运方案具体计算结果见附录A和附录B。

5.3.2 优化结果与灵敏度分析

(1) 结果对比

对加权多目标规划模型求解, 得到24周订购转运的最终方案, 各项关键运行指标总结在下表:

表 14 最终方案关键指标

项目	数值
总成本	670211.36
A类占比	32.00%
C类占比	28.00%
总损耗率	0.0200%
A类订货量	162821.12

C 类订销量	142468.48
总订销量	508816.00

从图中可以直观看出 A 类原料占比 32.00%，落在预设区间[24%,32%]，约束条件可以达成。C 类占比为 28.00%，低于预先设置的区间下限 28%-39%的下界。出现这一现象，说明多目标权衡之下，该模型为了追求更低损耗与更高 A 类占比，进一步压缩了 C 类原料采购规模。

将该优化方案与问题二成本最优的基准方案做横向对比，可以直观看到原料结构调整带来的综合改变，对比结果如下表所示：

表 15 问题二与问题三实施效果的对比

指标	问题二	问题三	变化
总成本	493999.08	670211.36	上升 35.7%
总订销量	430288.19	508816.00	上升 18.3%
A 类占比	0.0%	32.00%	上升 32.0 个百分点
C 类占比	63.2%	28.00%	下降 35.2 个百分点

从供应链结构来看，模型顺利完成多 A 少 C 的调整目标。基准方案完全不采购 A 类原料，优化之后 A 类占比达到 32%，C 类原料的依赖程度得到大幅削弱，原材料品类多元化程度明显提升。

结构优化需要承担对应的经济成本。总成本相比基准方案上涨 35.7%，粗略测算，A 类占比每提高 1 个百分点，总成本约增加 1.11%；C 类占比每下降 1 个百分点，总成本约增加 1.01%。总订销量同步上涨 18.3%，企业整体原料采购体量有所扩大。但转运环节收益十分明显，损耗率由基准的 0.57%下降至 0.02%，物料转运损失被控制在极低水平。

（2）权重灵敏度分析^[5]

模型权重经过熵权修正，存在一定经验成分，有必要检验权重扰动对输出结果的影响。将成本、A 类占比、损耗权重分别做+20%扰动，观察 A 类占比、总成本的波动情况，测试结果如下表所示

表 16 权重±20%扰动下灵敏度分析结果

实验	A 类占比(%)	C 类占比(%)	总成本	总损耗率(%)
原始	32	28	670211.4	0.02
成本+20%	29.38081	29.44055	666742.1	0.020348
A 类+20%	34.69718	26.51655	673075.5	0.020671
损耗+20%	31.93281	28.03695	670604.8	0.0187

权重发生 20%幅度变化时，A 类占比最大波动 2.70%，总成本最大波动仅 0.52%。即便权重出现较大偏移，方案关键指标没有发生剧烈震荡。整体来看模型输出具备较好稳健性，权重的人工修正没有造成结果过度敏感。

（3）综合评述

这套方案顺利实现提高 A 类原料占比、降低 C 类依赖的目标，转运损耗也控制在很低水平，但代价是采购总成本显著增加。现实生产决策中，企业需要在供应链安全与采购成本之间权衡取舍。

灵敏度的结果也可以给实际决策提供参考：如果企业更看重成本支出，可以适度降低 A 类占比目标，换取总成本下降；如果优先追求原料结构多元化，则可以接受当前的成本增幅，采纳该套优化方案。

5.4 问题四

经过第三问对供应商、转运商的联合优化，已经确定 28 家核心供应商与 8 家转运商的基础合作组合，得到 24 周基准订货-转运方案。在此基础上，企业计划通过内部技术改造挖掘生产潜力。技术改造完成后企业自身生产设备不再是产能约束，产能上限将由原料供应水平与转运运输能力共同决定。为求解技改条件下企业能够达到的最大成品产能，并给出与之匹配的订货、转运与库存方案，本文构建面向产能提升的混合整数优化模型^[6]。

5.4.1 核心数学模型的建立

(1) A、B、C 三类原料的供应上限

对于每一类原料 m ，该类别原料最大周供应能力为所有已选中该类供应商历史平均供货量累加：

$$\text{Cap}_m^{\text{supply}} = \sum_{i \in \mathcal{I}_m} \mu_i, m \in \{A, B, C\} \quad (46)$$

将原料供应上限结合原料消耗系数折算为对应的成品产能上限：

$$P_m^{\text{supply}} = \frac{\text{Cap}_m^{\text{supply}}}{r_m}, m \in \{A, B, C\} \quad (47)$$

(2) 运力动态优化分配

传统方式往往依托历史运输比例固定划分各类原料运力，但历史分配比例不一定适配产能提升后的需求场景。当某一类原料供货规模发生变化时，应当允许运力重新调配。本文将各类原料周运力 $v_{m,t}$ 设置为优化决策变量，运行程序根据各类原料实际订货需求自动完成运力分配，追求成品产能最大化。

8 家转运商合计总周运力上限为 48000m³，因此总运力约束：

$$\sum_{m \in \{A, B, C\}} v_{m,t} \leq 48000, \forall t \quad (48)$$

分配给某一类原料的运力不能超过该类别原料当周实际订货总量：

$$v_{m,t} \leq \sum_{i \in \mathcal{I}_m} x_{i,t}, \forall m, \forall t \quad (49)$$

分配到的运力能够支撑该类原料对应的成品产能：

$$\frac{v_{m,t}}{r_m} \geq P_m, \forall m, \forall t \quad (50)$$

(3) 各类原料最终产能贡献

单类原料能够实现的成品产能，同时受自身供货能力与分配得到的运输运力双重约束。原料供货充足但运力不足，原料无法运回工厂生产；运力充足但原料供货有限，同样无法释放产能。

$$P_m = \min\left(\frac{\text{Cap}_m^{\text{supply}}}{r_m}, \frac{v_m}{r_m}\right), m \in \{A, B, C\} \quad (51)$$

(4) 总产能上限最大化

模型第一阶段目标为求解企业技术改造之后可以实现的每周最大成品产能：

$$\max P = P_A + P_B + P_C \quad (52)$$

(5) 两周安全库存约束

按照题目库存管理规定，每一周结束，仓库原材料库存需要满足不少于两周生产消耗，约束表达式：

$$I_t \geq 2D, \forall t \quad (53)$$

式中 $D=P \times r$ 为每周原料总消耗量， r 代表 A、B、C 原料加权平均消耗系数。

5.4.2 瓶颈识别与供应商扩展

为定位限制产能进一步提高的核心因素，区分供应瓶颈与运力瓶颈两类情况，建立判断规则如下表示：

表 17 瓶颈类型判断条件

瓶颈类型	判断条件	结论
供应瓶颈	$\text{Cap}_m^{\text{supply}}/r_m < P_m$	第 m 类原料供应商供货能力不足，需要扩展该类别供应商
运力瓶颈	$v_m/r_m < P_m$ 且总运力 = 4.8 万	第 m 类原料运力不足，整体转运运力已经饱和

当模型识别出某一类原料存在供应瓶颈时，按照既定流程开展供应商扩充，具体实施步骤如下所示：

- 1) 从未选中的第 m 类供应商中，按问题一综合得分从高到低排序。
- 2) 依次将供应商纳入供货候选集和，重新计算该类别原料总供应能力 $\text{Cap}_m^{\text{supply}}$ 。
- 3) 满足 $\text{Cap}_m^{\text{supply}}/r_m \geq P_m$ 条件时停止新增供应商。
- 4) 记录新增供应商数量，测算新增供应商带来的总产能提升效果。

5.4.3 24 周订购与转运方案

求解得到企业最大可行产能 P 之后，把该产能作为新的生产需求输入混合整数线性规划（MILP）模型，复用前述模型框架求解未来 24 周的订货、转运与库存最优方案。

(1) 每周原料需求

产能确定后，每周生产所需原料总量： $D=P \times r$ ；其中 r 为 A、B、C 三类原料加权平均消耗系数，由各类原料实际消耗配比计算得到。

(2) 目标函数（总成本最小化）

该阶段优化目标为最小化 24 周期内企业综合成本，成本涵盖原料采购成本、库存持有成本以及货物运输损耗成本：

$$\min Z_2 = \sum_{i \in J^*} \sum_{t=1}^{24} c_i \cdot x_{i,t} + \sum_{t=1}^{24} h I_t + \sum_{i \in J^*} \sum_{k=1}^8 \sum_{t=1}^{24} c_i \frac{\lambda_{k,t}}{100} \cdot x_{i,t} \cdot u_{i,k,t} \quad (54)$$

(3) 约束条件

问题四模型建立的约束条件与问题二第二小问的约束条件相同。

5.4.4 模型的求解与结果分析

(1) 供应能力统计

提取第三问优化得到的 28 家合作供应商，按照 A、B、C 原料类别分别统计每周原料供应能力，并折算为对应成品产能，统计结果如下表所示：

表 18 各类原料供应商与供应能力汇总

原料类别	供应商数量	供应能力	折算成品产能
A 类	9	8074.85	13458.08
B 类	7	5494.68	8325.27
C 类	12	5863.1	8143.19
合计	28	19432.63	29926.54

由统计结果可知，28 家供应商保持历史平均供货水平条件下，折算成品产能合计 29926.54m³/周，该数值代表现有供应商组合理论最大供应上限。

(2) 运力优化求最大产能

以成品产能最大化为目标，将各类原料运力作为决策变量开展优化求解，总运力上限 48000m³/周，优化输出核心结果如表 19 所示：

表 19 产能优化求解结果

变量	数值
最大可行产能 P	29926.55m ³ /周
A 类运力分配 v _A	8074.85m ³ /周
B 类运力分配 v _B	5494.68m ³ /周
C 类运力分配 v _C	5863.10m ³ /周
总运力使用	19432.63/48000m ³ /周
原产能 P ₀	28200m ³ /周
产能提升幅度	1726.55m ³ /周
产能提升比例	6.12%

A、B、C 三类原料分别贡献成品产能 13458.08m³/周、8325.27m³/周、8143.19m³/周。优化后实际消耗运输运力仅 19432.63m³/周，转运资源存在大量剩余。对比原有 28200m³/周产能，技改之后最大可行产能提升 6.12%。

(3) 瓶颈识别

结合供应能力、运力使用情况开展瓶颈诊断，分析结果如表所示

表 20 瓶颈诊断结果

瓶颈类型	分析结果
供应能力	当前 28 家供应商总供应能力为 29926.54m ³ /周，与最大产能 29926.55m ³ /周基本相等，原料供应处于满负荷运行状态
运力能力	总运力使用为 19432.63m ³ /周，占 48000m ³ /周上限的 40.48%，转运运力富余充足
可扩展供应商	A、B、C 三类原料下无更高综合得分的待选供应商，没有新增供应商空间

综合判断，制约企业产能继续向上突破的核心瓶颈为供应商供货能力，而非转运运输环节，现有合作供应商已经接近供货能力天花板。

(4) 24 周订购与转运方案

将求解得到最大产能 $P=29926.55\text{m}^3/\text{周}$ 代入 MILP 模型，求解未来 24 周订货转运方案，关键参数以及输出指标如下表所示：

表 21 24 周方案关键指标

参数	数值
每周产能	$29926.55\text{m}^3/\text{周}$
每周原料需求	$29926.55 \times 0.66 \approx 19751.52\text{m}^3/\text{周}$
两周安全库存	39503.05m^3
总成本	509113.46 元
总订货量 (24 周)	459021.83m^3

模型输出 24 周详细订购方案、转运方案分别放置于附件 A、附件 B，全部周次期末库存均满足两周生产用量的安全库存硬性约束。

5.4.5 实施效果分析

将产能提升后的优化方案与初始产能 $28200\text{m}^3/\text{周}$ 基准方案进行横向对比，对比结果如表 22 所示：

表 22 产能提升前后方案对比

对比指标	原方案	新方案	变化
周产能	28200m^3	29926.55m^3	上升 6.12%
每周原料需求	18612m^3	19751.52m^3	上升 1139.52m^3
总订货量 (24 周)	430288.19m^3	459021.83m^3	上升 28733.64m^3
总成本	493999.08 元	509113.46 元	上升 15114.38 元
供应商数量	28 家	28 家	不变
转运商运力利用率	—	40.48%	充足富余
安全库存满足度	满足	满足	符合要求

由对比结果可以看出，在现有 28 家供应商组合约束下，企业产能提升空间有限，周产能最大提升幅度为 6.12%。各品类供应商均处于满负荷供货状态，无高评分备选供应商可补充，原料供给成为产能提升的主要限制条件。运输环节不存在资源约束，转运商现有运力可以完全承载产能提升后的全部运输任务，无需扩充转运商队伍。成本层面，方案总成本上涨 3.06%，成本增幅低于产能提升幅度，批量订货带来的规模效应有效摊薄单位产出成本。同时模型通过库存动态平衡约束保证各周期期末库存高于两周生产消耗量，满足题目规定的安全库存要求。

5.4.6 总结的结论

本文针对产能提升优化问题，基于 A/B/C 原料分类构建产能潜力分析模型，引入运力动态优化分配机制，完成瓶颈识别以及供应商扩展分析，求解得到 24 周期订货转运方案，得到核心结论如下：

(1) 选取已筛选的 28 家供应商，企业技术改造之后可以实现的周最大可产能为 $29926.55\text{m}^3/\text{周}$ ，对比原有 $28200\text{m}^3/\text{周}$ 产能提升 6.12%。

(2) 产能提升的主要制约因素是供应商原料供给能力，转运运输环节不存在瓶颈。当前供应商供货已经接近满负荷，没有高评分备选供应商可供新增；转运商整体运力利用率仅 40.48%，运输资源储备充足。

(3) 以最大可行产能作为生产需求，求解 24 周运营方案，方案总成本 509113.46 元，24 周合计原料订货量 $459021.83\text{m}^3/\text{周}$ ，全周期库存均满足两周生

产用量的安全库存约束。

六、模型的评价与推广

6.1 模型优点

1.评价体系客观严谨：构建四维供应商评价指标，经相关性分析剔除共线性指标，采用熵权-TOPSIS 模型完成综合排序，最大程度降低主观干扰，结果可真实反映供应商生产保障价值。

2.分层建模逻辑连贯：按照“筛选—最少组合—成本最优—结构调整—产能提升”递进建模，核心 MILP 框架与约束体系跨问题复用，模型一致性强，结果具备递进可比性。

3.多目标处理贴合实际：通过多场景生成备选方案，结合熵权法客观赋权；针对损耗率区分度不足问题进行权重修正，兼顾数据客观性与业务合理性；配套灵敏度分析验证了方案稳健性。

4.瓶颈识别定位精准：引入运力动态优化分配机制，建立供应与运力双维度瓶颈识别规则，可精准定位产能制约因素，配套供应商扩展策略为产能挖潜提供清晰路径。

6.2 模型局限性

1.供应商评价完全依赖历史数据，未纳入扩产潜力、应急响应等定性因素，对未来供货波动的预判能力有限。

2.损耗率权重修正带有一定经验判断色彩，适配本题决策导向，应用于其他损耗敏感场景时需重新校准。

6.3 模型推广

6.3.1 应用场景推广

整套模型可推广至各类制造企业的供应链管理场景：供应商评价模型可用于核心供应商筛选、供应风险评估；订购转运模型可用于采购计划制定、库存优化、转运调度；产能分析模型可用于企业技改规划、供应链瓶颈排查。

6.3.2 方法体系推广

“多场景生成—熵权赋权—经验修正—加权优化—灵敏度验证”的多目标处理框架，可推广至各类多目标权衡优化问题；“分类核算—动态分配—瓶颈识别”的产能分析思路，也可迁移至港口、仓储等其他资源约束下的产能优化场景。

七、AI 的使用说明

本参赛队使用了 AI 工具，主要用于梳理思路、代码调试、筛选关键词与文字润色，详细情况见支撑材料。

八、参考文献

- [1] 吕旻,万小鹏,师勇. 基于混合整数线性规划的飞行器防撞轨迹规划[J].飞行力学,2005,(2):81-84.DOI:10.13645/j.cnki.f.d.2005.02.021.
- [2] 程鹏,柳键. 改进的 TOPSIS 法在供应商选择中的应用研究[J].科技和产

业,2006,(4):39-40.

- [3] 连强. 综合区间数 Spearman 秩相关系数及其应用[J].重庆工商大学学报(自然科学版),2020,37(6):71-75.DOI:10.16055/j.issn.1672-058X.2020.0006.011.
- [4] 胡嘉峰,王丽丽. 基于熵权 AHP 修正 - TOPSIS 评价模型在选择方面的应用研究[J].湖南文理学院学报(自然科学版),2022,34(3):43-48+82.
- [5] 陈龙,朱颖,徐力,等. 基于权重灵敏度分析的结构形状等几何优化方法[J].中国机械工程,2020,31(11):1306-1314.
- [6] 陈娟,郑旺,刘倩倩,等. 基于动态储位分配策略的自动化立库多目标优化[J]. 系 统 仿 真 学 报 ,2025,37(6):1435-1448.DOI:10.16182/j.issn1004731x.joss.24-0210.

九、附录

支撑材料文件列表:

[..\..\..\..\Desktop\21 年 c 题支撑材料\402 家供应商供货特征与供货完成率计算结果表.xlsx](#)

[..\..\..\..\Desktop\21 年 c 题支撑材料\402 家供应商样本的均方误差.xlsx](#)

[..\..\..\..\Desktop\21 年 c 题支撑材料\供应商供货规模占比与物料集中度描述性统计.xlsx](#)

[..\..\..\..\Desktop\21 年 c 题支撑材料\全部供应商综合排名.xlsx](#)

[..\..\..\..\Desktop\21 年 c 题支撑材料\附件 A 订购方案数据结果\(2\)\(1\).xlsx](#)

[..\..\..\..\Desktop\21 年 c 题支撑材料\附件 B 转运方案数据结果\(2\)\(1\).xlsx](#)

[..\..\..\..\Desktop\21 年 c 题支撑材料\ai 使用说明.pdf](#)

代码:

不同周次下 ABC 三类原材料供货量变化趋势图 python 代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rcParams
rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'PingFang SC', 'Arial Unicode MS']
rcParams['axes.unicode_minus'] = False
supply_df = pd.read_excel(file_path, sheet_name="供应商的供货量 (m³)")
material_types = supply_df.iloc[:, 1].values
week_data = supply_df.iloc[:, 2:].values.astype(float)
a_supply = week_data[material_types == 'A'].sum(axis=0)
b_supply = week_data[material_types == 'B'].sum(axis=0)
c_supply = week_data[material_types == 'C'].sum(axis=0)
weeks = np.arange(1, 241)
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(weeks, a_supply, label='A 类', color='#2E86C1', linewidth=1.8)
ax.plot(weeks, b_supply, label='B 类', color='#E67E22', linewidth=1.8)
ax.plot(weeks, c_supply, label='C 类', color='#28B463', linewidth=1.8)
ax.set_xlabel('周次', fontsize=13)
ax.set_ylabel('供货量 (m³)', fontsize=13)
ax.set_title('不同周次下 A、B、C 三类原材料供货分布', fontsize=15, fontweight='bold')
ax.legend(loc='upper right', fontsize=12)
ax.grid(True, linestyle='--', alpha=0.6)
ax.ticklabel_format(axis='y', style='sci', scilimits=(0,0))
plt.tight_layout()
plt.savefig(output_png, dpi=300, bbox_inches='tight')
plt.savefig(output_pdf, bbox_inches='tight')
```

```
plt.show()
print(f'PNG 已保存: {output_png}')
print(f'PDF 已保存: {output_pdf}')
```

各供应商供货总量散点图 python 代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rcParams
rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
rcParams['axes.unicode_minus'] = False
supply_df = pd.read_excel(file_path, sheet_name="供应商的供货量 (m³)")
week_data = supply_df.iloc[:, 2:].values.astype(float)
total_supply = np.sum(week_data, axis=1)
n = len(total_supply)
supplier_index = np.arange(1, n + 1)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
ax1.scatter(supplier_index, total_supply, s=8, color='#2E86C1', alpha=0.6)
ax1.set_xlabel('供应商序号', fontsize=12)
ax1.set_ylabel('供货总量 (m³)', fontsize=12)
ax1.set_title('(a) 纵轴 0~500', fontsize=14, fontweight='bold')
ax1.set_ylim(0, 500)
ax1.grid(True, linestyle='--', alpha=0.4)
count_500 = np.sum(total_supply <= 500)
ax1.text(0.97, 0.95, f'0~500 区间: {count_500} 家',
        transform=ax1.transAxes, fontsize=11,
        verticalalignment='top', horizontalalignment='right',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))
ax2.scatter(supplier_index, total_supply, s=8, color='#E67E22', alpha=0.6)
ax2.set_xlabel('供应商序号', fontsize=12)
ax2.set_ylabel('供货总量 (m³)', fontsize=12)
ax2.set_title('(b) 纵轴 0~150', fontsize=14, fontweight='bold')
ax2.set_ylim(0, 150)
ax2.grid(True, linestyle='--', alpha=0.4)
count_150 = np.sum(total_supply <= 150)
ax2.text(0.97, 0.95, f'0~150 区间: {count_150} 家',
        transform=ax2.transAxes, fontsize=11,
        verticalalignment='top', horizontalalignment='right',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))
plt.tight_layout()
plt.savefig(output_png, dpi=300, bbox_inches='tight')
plt.savefig(output_pdf, bbox_inches='tight')
```

```

plt.show()
print(f'已保存: {output_png}')
print(f'已保存: {output_pdf}')
print("\n===== 统计信息 =====")
print(f'总供应商数: {n}')
print(f'供货量 0~500 的供应商: {count_500} 家")
print(f'供货量 0~150 的供应商: {count_150} 家")
print(f'供货量 > 500 的供应商: {n - count_500} 家")

```

相关性分析和供应商指标 python 代码

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib import rcParams
rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
rcParams['axes.unicode_minus'] = False
d1 = pd.read_excel(p1)
d2 = pd.read_excel(p2)
d = d1.merge(d2[['供应商 ID', '均方误差(MSE)']], on='供应商 ID', how='left')
d['供货规模占比'] = d.groupby('材料分类')['供货量总和'].transform(lambda
x: x / x.sum())
ct = d.groupby('材料分类')['供应商 ID'].count()
d['物料集中度'] = 1 / d['材料分类'].map(ct)
d['履约率'] = d['履约率']
d['稳定性_MSE'] = d['均方误差(MSE)']
d['连续性'] = d['供应次数总和'] / 240

cols_corr = ['供货规模占比', '履约率', '稳定性_MSE', '连续性', '物料集中度']
]
dfc = d[cols_corr].copy()

cs = dfc.corr(method='spearman')
cp = dfc.corr(method='pearson')

print("Spearman 相关系数矩阵")
print(cs.round(4))
print("\n 强相关检测 (|r|>0.7) ")
names = ['供货规模占比', '履约率', '稳定性_MSE', '连续性', '物料集中度']
for i in range(len(names)):
    for j in range(i+1, len(names)):
        r = cs.iloc[i, j]

```

```

        if abs(r) > 0.7:
            print(f' {names[i]} 与 {names[j]}: r={r:.4f} ')

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(cs, annot=True, fmt='.3f', cmap='RdBu_r',
            vmin=-1, vmax=1, center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8}, ax=ax)
ax.set_title('Spearman 相关系数热力图', fontsize=16, fontweight='bold')
plt.tight_layout()
cols_final = ['供应商 ID', '材料分类', '供货规模占比', '履约率', '稳定性_MSE', '物料集中度']
df_final = d[cols_final].copy()
df_final.to_excel(r"C:\Users\郭含梦\Desktop\数学建模论文\供应商指标对应表.xlsx", index=False)
print("\n 供应商指标对应表已保存")
print(f'共 {len(df_final)} 家供应商')

```

熵权法和 TOPSIS python 代码

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rcParams
rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
rcParams['axes.unicode_minus'] = False
df = pd.read_excel(p)
print(df.shape)
print(df.head(5))
cols = ['供货规模占比', '履约率', '稳定性_MSE', '物料集中度']
dirs = ['正向', '正向', '逆向', '正向']
X = df[cols].values.astype(float)
print(df[cols].describe())
def norm(X, dirs):
    Xn = np.zeros_like(X)
    for j in range(X.shape[1]):
        c = X[:, j]
        mn = c.min()
        mx = c.max()
        if dirs[j] == '正向':
            Xn[:, j] = (c - mn) / (mx - mn + 1e-10)
        else:
            Xn[:, j] = (mx - c) / (mx - mn + 1e-10)
    return Xn

```

```

Xn = norm(X, dirs)
print(Xn[:5].round(4))

def entropy(X):
    n, m = X.shape
    p = X / (X.sum(axis=0, keepdims=True) + 1e-10)
    e = -np.sum(p * np.log(p + 1e-10), axis=0) / np.log(n)
    w = (1 - e) / (1 - e).sum()
    return w

w = entropy(Xn)
for n, wv in zip(cols, w):
    print(f'{n}: {wv:.4f}')

def topsis(X, w):
    V = X * w
    Vp = V.max(axis=0)
    Vm = V.min(axis=0)
    Dp = np.sqrt(((V - Vp) ** 2).sum(axis=1))
    Dm = np.sqrt(((V - Vm) ** 2).sum(axis=1))
    C = Dm / (Dp + Dm + 1e-10)
    return C

s = topsis(Xn, w)
df['得分'] = s
df['排名'] = df['得分'].rank(ascending=False, method='dense').astype(int)
df_s = df.sort_values('排名').reset_index(drop=True)
print(df_s[['供应商 ID', '材料分类', '供货规模占比', '履约率', '稳定性_MSE',
'物料集中度', '得分']].head(20))
t50 = df_s.head(50)[['供应商 ID', '材料分类', '供货规模占比', '履约率', '稳定性_MSE', '物料集中度', '得分']]
print(t50['供应商 ID'].tolist())
print(t50['材料分类'].value_counts())
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(cols, w, color=['#2E86C1', '#E67E22', '#28B463', '#8E44AD'])
ax.set_ylabel('权重', fontsize=12)
ax.set_title('熵权法权重分布', fontsize=14, fontweight='bold')
ax.set_ylim(0, max(w) * 1.2)
for i, wv in enumerate(w):
    ax.text(i, wv + 0.02, f'{wv:.3f}', ha='center', va='bottom', fontsize=11)
plt.tight_layout()

```

求解最少供应商 python 代码

```
import pandas as pd
import numpy as np
print("=" * 70)
print("问题二：最少供应商数量求解")
print("=" * 70)
print("\n【第 1 步】读取数据...")
df_supply = pd.read_excel(PATH_附件 1, sheet_name='供应商的供货量 (m³)')

print(f"  供货量数据: {df_supply.shape[0]} 家, {df_supply.shape[1]-2} 周")
df_rank = pd.read_excel(PATH_综合排名)
print(f"  综合排名: {len(df_rank)} 家")
print("\n【第 2 步】计算正确的 mu_i (有订单时的平均周供货量) ...")
week_cols = df_supply.columns[2:]
mu_data = []
for idx, row in df_supply.iterrows():
    sid = row['供应商 ID']
    material = row['材料分类']
    supplies = row[week_cols].values.astype(float)

    positive = supplies[supplies > 0]

    if len(positive) > 0:
        mu = positive.mean()
        n_weeks = len(positive)
    else:
        mu = 0
        n_weeks = 0

    mu_data.append({
        '供应商 ID': sid,
        '材料分类': material,
        'mu_i': mu,
        '供货周数': n_weeks
    })
df_mu = pd.DataFrame(mu_data)
print(f"  计算完成: {len(df_mu)} 家")
print("\n【第 3 步】合并数据...")
df = pd.merge(df_rank, df_mu[['供应商 ID', 'mu_i']], on='供应商 ID',
              how='left')
df['mu_i'] = df['mu_i'].fillna(0)
print(f"  合并完成: {len(df)} 家")
```

```

print("\n【第 4 步】计算 E_i = mu_i / r_m ...")
r_m = {'A': 0.6, 'B': 0.66, 'C': 0.72}
df['r_m'] = df['材料分类'].map(r_m)
df['E_i'] = df['mu_i'] / df['r_m']
df = df.sort_values('排名').reset_index(drop=True)
total_E = df['E_i'].sum()
print(f" 全部 402 家累计 E_i: {total_E:.2f} m^3/周")
print("\n【第 5 步】贪心累加...")
df['累计 E_i'] = df['E_i'].cumsum()
THRESHOLD = 28200
print(f" 需求阈值: {THRESHOLD:,.2f} m^3/周")
df['达到阈值'] = df['累计 E_i'] >= THRESHOLD
first_row = df[df['达到阈值']].iloc[0] if not df[df['达到阈值']].empty else None
print("\n" + "=" * 70)
print("【结果输出】")
print("=" * 70)

if first_row is not None:
    idx = first_row.name
    n = idx + 1

    print(f"\n 最少供应商数量: {n} 家")
    print(f" 累计等效产能: {first_row['累计 E_i']:.2f} m^3/周")
    print(f" 需求阈值: {THRESHOLD:,.2f} m^3/周")
    print(f" 超出比例: {(first_row['累计 E_i'] / THRESHOLD - 1) * 100:.2f}%")

    df_selected = df.iloc[:n].copy()
    print(f"\n  【材料类别分布】")
    mat_stats = df_selected['材料分类'].value_counts()
    for m, c in mat_stats.items():
        print(f"    {m}类: {c} 家")
    print(f"\n  【前 20 家选中供应商】")
    print("-" * 90)
    print(f"{'排名':<6} {'供应商 ID':<12} {'材料':<6} {'mu_i':<12} {'E_i':<12} {'累计 E_i':<14}")
    print("-" * 90)
    for _, row in df_selected.head(20).iterrows():
        print(f"{'排名':<6} {row['供应商 ID']:<12} {row['材料分类']:<6} {row['mu_i']:<12.2f} {row['E_i']:<12.2f} {row['累计 E_i']:<14.2f}")

    if n > 20:
        print(f"  ... 共 {n} 家，以上仅显示前 20 家")

```

```

with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    df_selected.to_excel(writer, sheet_name='选中的供应商',
index=False)
    df.to_excel(writer, sheet_name='全部累加过程', index=False)

```

问题二 MILP 模型求解 python 代码

```

import pandas as pd
import pulp as pl
print("=" * 70)
print("第 2 步: MILP 模型求解 (订购与转运联合优化) - 修正版")
print("=" * 70)
df_suppliers = pd.read_excel(file_path, sheet_name='供应商列表')
df_loss = pd.read_excel(file_path, sheet_name='转运商损耗率', index_col=0)
df_params = pd.read_excel(file_path, sheet_name='固定参数')
selected = df_suppliers['供应商 ID'].tolist()
material_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['材料分类']))
mu_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['mu_i']))
if '历史最大供货量' in df_suppliers.columns:
    max_supply_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['历史
最大供货量']))
else:

    max_supply_dict = {row['供应商 ID']: row['mu_i'] * 2 for _, row in
df_suppliers.iterrows()}
    print("未找到'历史最大供货量'列, 使用 mu_i*2 作为上限")

price_dict = {'A': 1.2, 'B': 1.1, 'C': 1.0}
r_m = {'A': 0.6, 'B': 0.66, 'C': 0.72}
loss_rate = df_loss.values # shape: (8, 24)
alpha_fixed = {
    'A': 0.35,
    'B': 0.30,
    'C': 0.35
}
# 归一化
total_alpha = sum(alpha_fixed.values())
alpha_fixed = {k: v/total_alpha for k, v in alpha_fixed.items()}

# 计算每周各类材料消耗量
P = df_params[df_params['参数'] == '周产能(m3)']['数值'].values[0]
D_m_t = {}
for m in ['A', 'B', 'C']:

```

```

        D_m_t[m] = [P * alpha_fixed[m] * r_m[m] for t in range(24)]
# 每周总消耗量
D_t = [sum(D_m_t[m][t] for m in ['A', 'B', 'C']) for t in range(24)]
I0 = df_params[df_params['参数'] == '初始库存(m3)']['数值'].values[0]
M = df_params[df_params['参数'] == '运力上限(m3)']['数值'].values[0]
T = int(df_params[df_params['参数'] == '规划周期(周)']['数值'].values[0])
I = int(df_params[df_params['参数'] == '供应商数']['数值'].values[0])
K = int(df_params[df_params['参数'] == '转运商数']['数值'].values[0])
print(f"\n 周产能: {P} m³")
print(f" 初始库存: {I0} m³")
print(f" 运力上限: {M} m³/周")
print(f" 供应商数: {I} 家")
print(f" 转运商数: {K} 家")
print(f" 规划周期: {T} 周")
M_big = max(max_supply_dict.values()) * 2
print(f" Big-M: {M_big}")
SAFETY_WEEKS = 2
avg_consumption = sum(D_t) / T
SAFETY_STOCK = SAFETY_WEEKS * avg_consumption
print(f"\n 平均周消耗: {avg_consumption:.2f} m³")
print(f" 安全库存线: {SAFETY_STOCK:.2f} m³")
print(f" 库存上限: {8 * avg_consumption:.2f} m³")
print("\n 【1】 定义决策变量")
model = pl.LpProblem("Order_Transport_Plan", pl.LpMinimize)
# 订货量
x = pl.LpVariable.dicts("x", ((i, t) for i in range(I) for t in range(T)),
                        lowBound=0, cat='Continuous')

# 转运指派 (0-1 变量)
u = pl.LpVariable.dicts("u", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        cat='Binary')

# 实际运输量 (线性化辅助变量)
w = pl.LpVariable.dicts("w", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        lowBound=0, cat='Continuous')

# 期末库存
I_var = pl.LpVariable.dicts("I", (t for t in range(T)),
                        lowBound=0, cat='Continuous')

print(f" x_{i,t}: {I*T} 个")
print(f" u_{i,k,t}: {I*K*T} 个")

```

```

print(f"  w_{i,k,t}: {I*K*T} 个")
print(f"  I_t: {T} 个")
print("\n 【2】 构建目标函数")
# 采购成本
cost_purchase = pl.lpSum(
    price_dict[material_dict[selected[i]]] * x[(i, t)]
    for i in range(I) for t in range(T)
)
# 库存持有成本
cost_inventory = pl.lpSum(0.05 * I_var[t] for t in range(T))
# 运输损耗成本
cost_loss = pl.lpSum(
    price_dict[material_dict[selected[i]]] * (loss_rate[k, t] / 100) * w[(i, k, t)]
    for i in range(I) for k in range(K) for t in range(T)
)
model += cost_purchase + cost_inventory + cost_loss
print("  目标函数: 采购成本 + 库存成本 + 损耗成本")
print("\n 【3】 添加约束条件...")
constraint_count = 0
# 约束 1: 线性化  $w = x * u$ 
print("线性化约束 ( $w = x * u$ )")
for i in range(I):
    for k in range(K):
        for t in range(T):
            model += w[(i, k, t)] <= M_big * u[(i, k, t)]
            model += w[(i, k, t)] <= x[(i, t)]
            model += w[(i, k, t)] >= x[(i, t)] - M_big * (1 - u[(i, k, t)])
            model += w[(i, k, t)] >= 0
            constraint_count += 4

# 约束 2: 库存动态平衡 (使用固定  $D_t$ )
print("库存平衡方程")
for t in range(T):
    receive = pl.lpSum((1 - loss_rate[k, t] / 100) * w[(i, k, t)]
        for i in range(I) for k in range(K))
    if t == 0:
        model += I_var[t] == I0 + receive - D_t[t]
    else:
        model += I_var[t] == I_var[t-1] + receive - D_t[t]
    constraint_count += 1

# 约束 3: 安全库存 (使用固定安全库存线)

```

```

print("安全库存约束")
for t in range(T):
    # 下限：至少保留 2 周平均消耗量的库存
    model += I_var[t] >= SAFETY_STOCK
    # 上限：不超过 8 周平均消耗量
    model += I_var[t] <= 8 * avg_consumption
    constraint_count += 2
# 约束 4：供应商供货能力上限
print("供应商能力约束")
for i in range(I):
    max_supply = max_supply_dict[selected[i]]
    for t in range(T):
        model += x[(i, t)] <= max_supply
        constraint_count += 1

# 约束 5：转运指派唯一性
print("转运指派唯一性")
for i in range(I):
    for t in range(T):
        model += pl.lpSum(u[(i, k, t)] for k in range(K)) == 1
        constraint_count += 1

# 约束 6：转运商运力上限
print("转运商运力约束")
for k in range(K):
    for t in range(T):
        model += pl.lpSum(w[(i, k, t)] for i in range(I)) <= M
        constraint_count += 1

print(f" 共添加 {constraint_count} 个约束")
print("\n【4】求解 MILP 模型...")
print(" 求解器: PuLP (CBC)")
print(" 时间限制: 300 秒")
print(" 请稍候...")

model.solve(pl.PULP_CBC_CMD(msg=True, timeLimit=300, gapRel=0.01))
print("\n" + "=" * 70)
print("求解结果")
print("=" * 70)

status = pl.LpStatus[model.status]
print(f" 求解状态: {status}")

```

```

if model.status == 1: # Optimal
    print(f'找到最优解！')

    total_cost = pl.value(model.objective)
    print(f'\n 总成本: {total_cost:.2f}')
    print(f' 采购成本: {pl.value(cost_purchase):.2f}')
    print(f' 库存成本: {pl.value(cost_inventory):.2f}')
    print(f' 损耗成本: {pl.value(cost_loss):.2f}')
    print("\n【5】导出订购和转运方案...")

    # 统计非零订货量
    non_zero = 0
    total_order = 0
    for i in range(I):
        for t in range(T):
            val = pl.value(x[(i, t)])
            if val > 0.01:
                non_zero += 1
                total_order += val
    print(f' 非零订货量记录数: {non_zero} / {I*T}')
    print(f' 总订货量: {total_order:.2f} m³')

    # 导出订购方案
    results = []
    for i in range(I):
        for t in range(T):
            assigned_k = None
            for k in range(K):
                if pl.value(u[(i, k, t)]) > 0.5:
                    assigned_k = k
                    break
            if assigned_k is not None:
                results.append({
                    '周次': t+1,
                    '供应商': selected[i],
                    '材料': material_dict[selected[i]],
                    '订货量': pl.value(x[(i, t)]),
                    '转运商': f'T{assigned_k+1}',
                    '损耗率(%)': loss_rate[assigned_k, t]
                })

```

```

df_plan = pd.DataFrame(results)

# 导出库存变化
inventory = []
for t in range(T):
    inventory.append({
        '周次': t+1,
        '期末库存': pl.value(I_var[t]),
        '本周消耗': D_t[t],
        '安全库存线': SAFETY_STOCK,
        '是否低于安全线': '是' if pl.value(I_var[t]) < SAFETY_STOCK
    else '否'
    })
df_inv = pd.DataFrame(inventory)

# 保存到 Excel
output_file = r"C:\Users\郭含梦\Desktop\数学建模论文\订购和转运方案.xlsx"
with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    df_plan.to_excel(writer, sheet_name='订购与转运方案',
index=False)
    df_inv.to_excel(writer, sheet_name='库存变化', index=False)

print(f"  已保存至: {output_file}")

else:
    print(f"求解失败！ ")
    print("\n  可能原因: ")
    print("    1. 初始库存仍不足")
    print("    2. 安全库存线设置过高")
    print("    3. 供应商总产能不足")
    print("    4. 转运商运力不足")

# 导出 LP 文件用于调试
model.writeLP("debug.lp")
print("\n  已导出 debug.lp 文件，可用文本编辑器打开查看约束")
print("  请将该文件发送给我，我帮你定位具体冲突")

print("\n" + "=" * 70)
print("第 2 步求解完成！ ")
print("=" * 70)

```

问题二 方案实施效果分析 python 代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import warnings
warnings.filterwarnings('ignore')

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial
Unicode MS']
matplotlib.rcParams['axes.unicode_minus'] = False
print("=" * 70)
print("问题二：实施效果分析")
print("=" * 70)
df_plan = pd.read_excel(file_result, sheet_name='订购与转运方案')
df_inv = pd.read_excel(file_result, sheet_name='库存变化')

print("\n【1】库存水位分析")
print("=" * 70)

I_t = df_inv['期末库存'].values
D_t = df_inv['本周消耗'].values

P = 28200
r_B = 0.66
D = P * r_B
SAFETY_STOCK = 2 * D

all_safe = all(I_t >= SAFETY_STOCK)
min_inv = min(I_t)
min_week = np.argmin(I_t) + 1
max_inv = max(I_t)
max_week = np.argmax(I_t) + 1
avg_inv = np.mean(I_t)
safety_margin = min_inv - SAFETY_STOCK

print(f" 安全库存线: {SAFETY_STOCK:.2f} m³")
print(f" 最低库存: 第{min_week}周 ({min_inv:.2f} m³)")
print(f" 最高库存: 第{max_week}周 ({max_inv:.2f} m³)")
print(f" 平均库存: {avg_inv:.2f} m³")
print(f" 安全余量: {safety_margin:.2f} m³")
```

```

print(f" 全部周次满足安全库存: {'是' if all_safe else '否'}")

fig, ax = plt.subplots(figsize=(12, 6))
weeks = list(range(1, 25))
ax.plot(weeks, I_t, marker='o', linewidth=2, markersize=6, label='期末库存',
color='steelblue')
ax.axhline(y=SAFETY_STOCK, color='red', linestyle='--', linewidth=2,
label=f'安全库存线 = {SAFETY_STOCK:.0f} m³')
ax.fill_between(weeks, SAFETY_STOCK, max(I_t)*1.1, alpha=0.1,
color='green')
ax.set_xlabel('周次', fontsize=12)
ax.set_ylabel('库存量 (m³)', fontsize=12)
ax.set_title('未来 24 周库存变化趋势', fontsize=14, fontweight='bold')
ax.legend(loc='best')
ax.grid(True, alpha=0.3)
ax.set_xticks(weeks)
ax.annotate(f'最低库存: {min_inv:.0f} m³\n(第 {min_week} 周)',
            xy=(min_week, min_inv),
            xytext=(min_week+3, min_inv + 5000),
            arrowprops=dict(arrowstyle='->', color='red'),
            fontsize=10, color='red')

plt.tight_layout()
print("\n" + "=" * 70)
print(" 【2】 成本构成分析")
print("=" * 70)
total_cost = 493999.08
cost_purchase = 446118.57
cost_inventory = 45056.04
cost_loss = 2824.48

ratio_purchase = cost_purchase / total_cost * 100
ratio_inventory = cost_inventory / total_cost * 100
ratio_loss = cost_loss / total_cost * 100

print(f" 总成本: {total_cost:,.2f} 元")
print(f" 采购成本: {cost_purchase:,.2f} 元 ({ratio_purchase:.2f}%)")
print(f" 库存成本: {cost_inventory:,.2f} 元 ({ratio_inventory:.2f}%)")
print(f" 损耗成本: {cost_loss:,.2f} 元 ({ratio_loss:.2f}%)")

fig2, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
labels = ['采购成本', '库存成本', '损耗成本']
sizes = [ratio_purchase, ratio_inventory, ratio_loss]

```

```

colors = ['#ff6b6b', '#4ecdc4', '#ffe66d']
explode = (0.05, 0.05, 0.05)

ax1.pie(sizes, explode=explode, labels=labels, colors=colors,
autopct='%1.2f%%', shadow=True, startangle=90)
ax1.set_title('成本构成占比', fontsize=14, fontweight='bold')

x_pos = range(3)
ax2.bar(x_pos, sizes, color=colors, width=0.6)
ax2.set_xticks(x_pos)
ax2.set_xticklabels(labels)
ax2.set_ylabel('占比 (%)', fontsize=12)
ax2.set_title('成本构成柱状图', fontsize=14, fontweight='bold')
ax2.set_ylim(0, 105)
for i, v in enumerate(sizes):
    ax2.text(i, v + 1, f'{v:.2f}%', ha='center', va='bottom', fontsize=11,
fontweight='bold')
plt.tight_layout()
print(f'  成本构成图已保存')
print("\n" + "=" * 70)
print(" 【3】 供应商采购分析")
print("=" * 70)

supplier_order = df_plan.groupby(' 供 应 商 ')[ ' 订 货 量
'].sum().sort_values(ascending=False)
supplier_material = df_plan.groupby('供应商')['材料'].first()

print("  前 10 大供应商:")
for i, (sid, val) in enumerate(supplier_order.head(10).items()):
    mat = supplier_material[sid]
    print(f'    {i+1}. {sid} ({mat}): {val:.2f} m³')

mat_order = df_plan.groupby('材料')['订货款'].sum()
total_order = mat_order.sum()
print(f'\n  各材料采购量:')
for mat in ['A', 'B', 'C']:
    if mat in mat_order.index:
        print(f'    {mat}:    {mat_order[mat]:.2f}    m³
({mat_order[mat]/total_order*100:.2f}%)')

fig3, ax = plt.subplots(figsize=(10, 6))
top15 = supplier_order.head(15)

```

```

colors_top = ['#ff6b6b' if supplier_material[sid] == 'A' else '#4ecdc4' if
supplier_material[sid] == 'B' else '#ffe66d' for sid in top15.index]
ax.barh(range(len(top15)), top15.values, color=colors_top)
ax.set_yticks(range(len(top15)))
ax.set_yticklabels(top15.index, fontsize=10)
ax.set_xlabel('总订货量 (m³)', fontsize=12)
ax.set_title('供应商订货量排名 (前 15)', fontsize=14, fontweight='bold')
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#ff6b6b', label='A 类'),
                    Patch(facecolor='#4ecdc4', label='B 类'),
                    Patch(facecolor='#ffe66d', label='C 类')]
ax.legend(handles=legend_elements, loc='lower right')
plt.tight_layout()
plt.savefig(r"C:\Users\郭含梦\Desktop\数学建模论文\供应商分析图.png",
dpi=300, bbox_inches='tight')
print(f"  供应商分析图已保存")
report = f"""
问题二 实施效果分析报告
1. 库存水位分析
    安全库存线: {SAFETY_STOCK:.0f} m³
    最低库存周: 第{min_week}周 ({min_inv:.0f} m³)
    安全余量: {safety_margin:.0f} m³
    全部周次满足安全库存: {'是' if all_safe else '否'}

2. 成本构成分析
    总成本: {total_cost:,.2f} 元
    采购成本: {cost_purchase:,.2f} 元 ({ratio_purchase:.2f}%)
    库存成本: {cost_inventory:,.2f} 元 ({ratio_inventory:.2f}%)
    损耗成本: {cost_loss:,.2f} 元 ({ratio_loss:.2f}%)

3. 供应商采购分析
    总订货量: {total_order:,.2f} m³
    最大采购供应商: {supplier_order.index[0]}
    ({supplier_order.values[0]:,.2f} m³)
    各材料占比: A {mat_order.get('A', 0)/total_order*100:.1f}%,
                B {mat_order.get('B', 0)/total_order*100:.1f}%,
                C {mat_order.get('C', 0)/total_order*100:.1f}%

4. 总体结论
    该方案在 24 周规划期内:
        1. 库存水平始终保持在安全库存线以上, 最低安全余量
        {safety_margin:.0f} m³;

```

2. 总成本 {total_cost:,.2f} 元, 采购成本占比 {ratio_purchase:.1f}%, 成本结构合理;

3. 供应商分布均衡, 能够满足企业正常生产需求。

综合评价: 方案可行, 具备良好的经济性和可执行性。

问题三 探索 A 类占比约束求解 python 代码

```
import pandas as pd
```

```
import pulp as pl
```

```
print("=" * 70)
```

```
print("问题三: MILP 模型求解 (A 类占比约束场景)")
```

```
print("=" * 70)
```

```
file_path = r"C:\Users\郭含梦\Desktop\数学建模论文\订购和转运模型所需数据.xlsx"
```

```
supply_file = r"C:\Users\郭含梦\Desktop\数学建模论文\附件 1 近 5 年 402 家供应商的相关数据.xlsx"
```

```
df_suppliers = pd.read_excel(file_path, sheet_name='供应商列表')
```

```
df_loss = pd.read_excel(file_path, sheet_name='转运商损耗率', index_col=0)
```

```
df_params = pd.read_excel(file_path, sheet_name='固定参数')
```

```
print("\n【1】从附件 1 统计历史最大供货量...")
```

```
df_supply_raw = pd.read_excel(supply_file, sheet_name='供应商的供货量 (m³)')
```

```
supply_cols = df_supply_raw.columns[2:]
```

```
real_max_supply = {}
```

```
for idx, row in df_supply_raw.iterrows():
```

```
    sid = row.iloc[0]
```

```
    max_val = row[supply_cols].max()
```

```
    real_max_supply[sid] = max_val
```

```
print(f"  已统计 {len(real_max_supply)} 家供应商的历史最大供货量")
```

```
selected = df_suppliers['供应商 ID'].tolist()
```

```
material_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['材料分类']))
```

```
mu_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['mu_i']))
```

```
max_supply_dict = {}
```

```
for sid in selected:
```

```
    if sid in real_max_supply and real_max_supply[sid] > 0:
```

```
        max_supply_dict[sid] = real_max_supply[sid]
```

```
    else:
```

```
        max_supply_dict[sid] = mu_dict.get(sid, 0) * 2
```

```
    if sid not in real_max_supply:
```

```
        print(f"  警告: {sid} 未在附件 1 中找到, 使用 mu_i*2")
```

```

        elif real_max_supply[sid] == 0:
            print(f"  警告: {sid} 历史最大供货量为 0, 使用 mu_i*2")

price_dict = {'A': 1.2, 'B': 1.1, 'C': 1.0}
r_m = {'A': 0.6, 'B': 0.66, 'C': 0.72}
loss_rate = df_loss.values
alpha_fixed = {'A': 0.35, 'B': 0.30, 'C': 0.35}
total_alpha = sum(alpha_fixed.values())
alpha_fixed = {k: v/total_alpha for k, v in alpha_fixed.items()}
P = df_params[df_params['参数'] == '周产能(m3)']['数值'].values[0]
D_m_t = {}
for m in ['A', 'B', 'C']:
    D_m_t[m] = [P * alpha_fixed[m] * r_m[m] for t in range(24)]
D_t = [sum(D_m_t[m][t] for m in ['A', 'B', 'C']) for t in range(24)]
I0 = df_params[df_params['参数'] == '初始库存(m3)']['数值'].values[0]
M = df_params[df_params['参数'] == '运力上限(m3)']['数值'].values[0]
T = int(df_params[df_params['参数'] == '规划周期(周)']['数值'].values[0])
I = int(df_params[df_params['参数'] == '供应商数']['数值'].values[0])
K = int(df_params[df_params['参数'] == '转运商数']['数值'].values[0])

print(f"\n  周产能: {P} m³")
print(f"  初始库存: {I0} m³")
print(f"  运力上限: {M} m³/周")
print(f"  供应商数: {I} 家")
print(f"  转运商数: {K} 家")
print(f"  规划周期: {T} 周")

M_big = max(max_supply_dict.values()) * 2
print(f"  Big-M: {M_big:.2f}")
SAFETY_WEEKS = 2
avg_consumption = sum(D_t) / T
SAFETY_STOCK = SAFETY_WEEKS * avg_consumption
print(f"\n  平均周消耗: {avg_consumption:.2f} m³")
print(f"  安全库存线: {SAFETY_STOCK:.2f} m³")
print(f"  库存上限: {8 * avg_consumption:.2f} m³")

theta_A = 0.24
theta_C = 0.39
scene_name = f" 场 景"
print(f"{int(theta_A*100/3)}_A{int(theta_A*100)}%_C{int(theta_C*100)}%")

print(f"\n{' '*70}")

```

```

print(f"当前场景: {scene_name}")
print(f"  A 类占比 >= {theta_A*100}%")
print(f"  C 类占比 <= {theta_C*100}%")
print(f'{'='*70}')

print("\n 【2】 定义决策变量...")

model = pl.LpProblem("Order_Transport_Plan_Problem3", pl.LpMinimize)

x = pl.LpVariable.dicts("x", ((i, t) for i in range(I) for t in range(T)),
                        lowBound=0, cat='Continuous')

u = pl.LpVariable.dicts("u", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        cat='Binary')

w = pl.LpVariable.dicts("w", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        lowBound=0, cat='Continuous')

I_var = pl.LpVariable.dicts("I", (t for t in range(T)),
                        lowBound=0, cat='Continuous')

print(f"  x_{i,t}: {I*T} 个")
print(f"  u_{i,k,t}: {I*K*T} 个")
print(f"  w_{i,k,t}: {I*K*T} 个")
print(f"  I_t: {T} 个")

print("\n 【3】 构建目标函数...")

cost_purchase = pl.lpSum(
    price_dict[material_dict[selected[i]]] * x[(i, t)]
    for i in range(I) for t in range(T)
)

cost_inventory = pl.lpSum(0.05 * I_var[t] for t in range(T))

cost_loss = pl.lpSum(
    price_dict[material_dict[selected[i]]] * (loss_rate[k, t] / 100) * w[(i, k, t)]
    for i in range(I) for k in range(K) for t in range(T)
)

```

```

model += cost_purchase + cost_inventory + cost_loss

print("  目标函数: 采购成本 + 库存成本 + 损耗成本")

print("\n 【4】 添加约束条件...")
constraint_count = 0

print("  - 线性化约束...")
for i in range(I):
    for k in range(K):
        for t in range(T):
            model += w[(i, k, t)] <= M_big * u[(i, k, t)]
            model += w[(i, k, t)] <= x[(i, t)]
            model += w[(i, k, t)] >= x[(i, t)] - M_big * (1 - u[(i, k, t)])
            model += w[(i, k, t)] >= 0
            constraint_count += 4

print("  - 库存平衡方程...")
for t in range(T):
    receive = pl.lpSum((1 - loss_rate[k, t] / 100) * w[(i, k, t)]
                        for i in range(I) for k in range(K))
    if t == 0:
        model += I_var[t] == I0 + receive - D_t[t]
    else:
        model += I_var[t] == I_var[t-1] + receive - D_t[t]
    constraint_count += 1

print("  - 安全库存约束...")
for t in range(T):
    model += I_var[t] >= SAFETY_STOCK
    model += I_var[t] <= 8 * avg_consumption
    constraint_count += 2

print("  - 供应商能力约束...")
for i in range(I):
    max_supply = max_supply_dict[selected[i]]
    for t in range(T):
        model += x[(i, t)] <= max_supply
        constraint_count += 1

print("  - 转运指派唯一性...")
for i in range(I):

```

```

        for t in range(T):
            model += pl.lpSum(u[(i, k, t)] for k in range(K)) == 1
            constraint_count += 1

print(" - 转运商运力约束...")
for k in range(K):
    for t in range(T):
        model += pl.lpSum(w[(i, k, t)] for i in range(I)) <= M
        constraint_count += 1

print(" - A/C 占比约束...")

A_total = pl.lpSum(x[(i, t)] for i in range(I) for t in range(T)
                    if material_dict[selected[i]] == 'A')
C_total = pl.lpSum(x[(i, t)] for i in range(I) for t in range(T)
                    if material_dict[selected[i]] == 'C')
total_order = pl.lpSum(x[(i, t)] for i in range(I) for t in range(T))

model += A_total >= theta_A * total_order
model += C_total <= theta_C * total_order
constraint_count += 2

print(f"    A 类占比 >= {theta_A*100}%")
print(f"    C 类占比 <= {theta_C*100}%")
print(f"\n  共添加 {constraint_count} 个约束")

print("\n【5】求解 MILP 模型...")
print("  求解器: PuLP (CBC)")
print("  时间限制: 300 秒")
print("  请稍候...")

model.solve(pl.PULP_CBC_CMD(msg=True, timeLimit=300, gapRel=0.01))

print("\n" + "=" * 70)
print("【求解结果】")
print("=" * 70)

status = pl.LpStatus[model.status]
print(f"  求解状态: {status}")

if model.status == 1:
    print(f"  找到最优解!")

```

```

total_cost = pl.value(model.objective)
A_val = pl.value(A_total)
C_val = pl.value(C_total)
total_val = pl.value(total_order)
A_ratio = A_val / total_val if total_val > 0 else 0
C_ratio = C_val / total_val if total_val > 0 else 0
loss_val = pl.value(cost_loss)
loss_rate_total = loss_val / total_val if total_val > 0 else 0

print(f"\n 总成本: {total_cost:.2f}")
print(f" 采购成本: {pl.value(cost_purchase):.2f}")
print(f" 库存成本: {pl.value(cost_inventory):.2f}")
print(f" 损耗成本: {pl.value(cost_loss):.2f}")
print(f"\n  【A/C 占比统计】 ")
print(f"    A 类总订货量: {A_val:.2f} m³")
print(f"    C 类总订货量: {C_val:.2f} m³")
print(f"    总订货量: {total_val:.2f} m³")
print(f"    A 类占比: {A_ratio*100:.2f}%")
print(f"    C 类占比: {C_ratio*100:.2f}%")
print(f"    总损耗率: {loss_rate_total*100:.2f}%")

with open(f'{scene_name}_目标值.txt', 'w', encoding='utf-8') as f:
    f.write(f'场景: {scene_name}\n')
    f.write(f'theta_A: {theta_A}\n')
    f.write(f'theta_C: {theta_C}\n')
    f.write(f'总成本: {total_cost:.2f}\n')
    f.write(f'A 类占比: {A_ratio*100:.2f}%\n')
    f.write(f'C 类占比: {C_ratio*100:.2f}%\n')
    f.write(f'总损耗率: {loss_rate_total*100:.2f}%\n')
    f.write(f'A 类订货量: {A_val:.2f}\n')
    f.write(f'C 类订货量: {C_val:.2f}\n')
    f.write(f'总订货量: {total_val:.2f}\n')

print("\n 【6】 导出订购和转运方案...")

results = []
for i in range(I):
    for t in range(T):
        x_val = pl.value(x[(i, t)])
        if x_val > 0.01:
            assigned_k = None

```

```

        for k in range(K):
            if pl.value(u[(i, k, t)]) > 0.5:
                assigned_k = k
                break
        if assigned_k is not None:
            results.append({
                '周次': t+1,
                '供应商': selected[i],
                '材料': material_dict[selected[i]],
                '订货量': x_val,
                '转运商': fT{assigned_k+1}',
                '损耗率(%)': loss_rate[assigned_k, t]
            })

df_plan = pd.DataFrame(results)

inventory = []
for t in range(T):
    inventory.append({
        '周次': t+1,
        '期末库存': pl.value(I_var[t]),
        '本周消耗': D_t[t],
        '安全库存线': SAFETY_STOCK,
        '是否低于安全线': '是' if pl.value(I_var[t]) < SAFETY_STOCK
    else '否'
    })
df_inv = pd.DataFrame(inventory)

output_file = f'{scene_name}_订购转运方案.xlsx'
with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    df_plan.to_excel(writer, sheet_name='订购与转运方案',
index=False)
    df_inv.to_excel(writer, sheet_name='库存变化', index=False)

print(f"  已保存至: {output_file}")

print("\n" + "=" * 70)
print(" 【场景关键指标汇总】 ")
print("=" * 70)
print(f"   A 类 占比 约束 : >= {theta_A*100}%    ->   实际 :
{A_ratio*100:.2f}%")
print(f"   C 类 占比 约束 : <= {theta_C*100}%    ->   实际 :

```

```

{C_ratio*100:.2f}%")
    print(f"  总成本: {total_cost:.2f}")
    print(f"  总损耗率: {loss_rate_total*100:.2f}%")
    print("=" * 70)

else:
    print(f"  求解失败! 状态: {status}")
    print("\n  可能原因: ")
    print("      1. A 类占比目标过高, A 类供应商产能不足")
    print("      2. C 类占比限制过紧, C 类供应商被过度压缩")
    print("      3. 安全库存与 A/C 约束冲突")

    model.writeLP(f'{scene_name}_debug.lp')
    print(f"\n  已导出 {scene_name}_debug.lp 文件")

print("\n" + "=" * 70)
print(f'{scene_name} 求解完成! ')
print("=" * 70)

```

问题三 熵权法求权重 python 代码

```

import pandas as pd
import numpy as np

print("=" * 70)
print("熵权法计算（修正权重：损耗率固定 10%，其余动态分配）")
print("=" * 70)

data = {
    '方案': ['场景 1(15%)', '场景 2(18%)', '场景 3(21%)', '场景 4(24%)'],
    '总成本': [501523.29, 504123.82, 506657.65, 509155.53],
    'A 类占比': [15.00, 18.00, 21.00, 24.00],
    'C 类占比': [48.00, 45.00, 42.00, 39.00],
    '损耗率': [0.08, 0.08, 0.08, 0.07]
}

df = pd.DataFrame(data)
print("\n 原始数据")
print(df.to_string(index=False))

X = df.iloc[:, 1:].values.astype(float)
n_rows, n_cols = X.shape

```

```

print(f"\n 方案数: {n_rows}, 指标数: {n_cols}")

print("\n 数据标准化")

directions = {
    '总成本': 0,
    'A 类占比': 1,
    'C 类占比': 0,
    '损耗率': 0
}

X_norm = np.zeros((n_rows, n_cols))
for j in range(n_cols):
    col_name = df.columns[j+1]
    col_min = X[:, j].min()
    col_max = X[:, j].max()
    if directions[col_name] == 1:
        X_norm[:, j] = (X[:, j] - col_min) / (col_max - col_min + 1e-10)
    else:
        X_norm[:, j] = (col_max - X[:, j]) / (col_max - col_min + 1e-10)

df_norm = pd.DataFrame(X_norm, columns=df.columns[1:])
df_norm.insert(0, '方案', df['方案'])
print(df_norm.to_string(index=False))

print("\n 计算特征比重")

p = np.zeros((n_rows, n_cols))
for j in range(n_cols):
    col_sum = X_norm[:, j].sum()
    if col_sum > 0:
        p[:, j] = X_norm[:, j] / col_sum
    else:
        p[:, j] = 1.0 / n_rows

df_p = pd.DataFrame(p, columns=df.columns[1:])
df_p.insert(0, '方案', df['方案'])
print(df_p.to_string(index=False))

print("\n 计算熵值 e_j 和 差异系数 d_j")

k = 1.0 / np.log(n_rows)

```

```

e = np.zeros(n_cols)
d = np.zeros(n_cols)

for j in range(n_cols):
    p_col = p[:, j]
    log_p = np.log(p_col + 1e-12)
    e[j] = -k * np.sum(p_col * log_p)
    d[j] = 1 - e[j]

for j, col_name in enumerate(df.columns[1:]):
    print(f"    {col_name}: e = {e[j]:.6f}, d = {d[j]:.6f}")

print("\n 计算原始熵权")

d_sum = d.sum()
w_original = d / d_sum

print("\n  原始熵权法权重: ")
for j, col_name in enumerate(df.columns[1:]):
    print(f"                                {col_name}:          {w_original[j]:.4f}
({w_original[j]*100:.2f}%)")

print("\n 修正权重（损耗率固定 10%，其余动态分配）")

w4_fixed = 0.10
w1_original = w_original[0]
w2_original = w_original[1]
w3_original = w_original[2]
total_original = w1_original + w2_original + w3_original

w1 = (w1_original / total_original) * (1 - w4_fixed)
w2 = (w2_original / total_original) * (1 - w4_fixed)
w3 = (w3_original / total_original) * (1 - w4_fixed)
w4 = w4_fixed

print(f"  损耗率固定为: {w4_fixed*100:.0f}%")
print("\n  修正后权重: ")
print(f"    总成本: {w1:.4f} ({w1*100:.2f}%)")
print(f"    A 类占比: {w2:.4f} ({w2*100:.2f}%)")
print(f"    C 类占比: {w3:.4f} ({w3*100:.2f}%)")
print(f"    损耗率: {w4:.4f} ({w4*100:.2f}%)")

```

```

print("\n 计算各方案综合得分")

scores = np.sum(X_norm * np.array([w1, w2, w3, w4]), axis=1)
scores_norm = scores / scores.max() * 100

print("\n 综合得分: ")
for i, row in df.iterrows():
    print(f"      {row['方案']}: {scores[i]:.6f} (归一化: {scores_norm[i]:.2f}分)")

print("\n 推荐方案")

df_result = df.copy()
df_result['综合得分'] = scores
df_result['归一化得分'] = scores_norm
df_result = df_result.sort_values('综合得分', ascending=False)

print("\n 方案排序 (从优到劣): ")
for idx, row in df_result.iterrows():
    rank = df_result.index.get_loc(idx) + 1
    print(f"      第 {rank} 名: {row['方案']} (得分: {row['归一化得分']:.2f})")
    best = df_result.iloc[0]
    print(f"\n 最优方案: {best['方案']}")
    print(f"      A 类占比: {best['A 类占比']:.0f}%")
    print(f"      C 类占比: {best['C 类占比']:.0f}%")
    print(f"      总成本: {best['总成本']:.2f}")
    print(f"      总损耗率: {best['损耗率']:.2f}%")
    output = {
        '指标': df.columns[1:].tolist(),
        '方向': ['负向' if directions[col] == 0 else '正向' for col in df.columns[1:]],
        '原始熵值_e': e.tolist(),
        '原始差异系数_d': d.tolist(),
        '原始权重_w': w_original.tolist(),
        '修正权重_w': [w1, w2, w3, w4]
    }
df_output = pd.DataFrame(output)
print("\n 【权重汇总表】 ")
print(df_output.to_string(index=False))

with pd.ExcelWriter('熵权法结果_修正权重.xlsx', engine='openpyxl') as writer:

```

```

df_norm.to_excel(writer, sheet_name='标准化数据', index=False)
df_p.to_excel(writer, sheet_name='特征比重', index=False)
df_output.to_excel(writer, sheet_name='熵权法结果', index=False)
df_result[['方案', '总成本', 'A 类占比', 'C 类占比', '损耗率', '综合得分', '
归一化得分']].to_excel(
    writer, sheet_name='综合得分排序', index=False
)
print("\n 已导出: 熵权法结果_修正权重.xlsx")
print("\n" + "=" * 70)
print("熵权法计算完成!")
print("=" * 70)

```

问题三 MILP 求方案 python 代码

```

import pandas as pd
import pulp as pl

print("=" * 70)
print("问题三: 最终方案求解 (修正权重 + 四边硬约束)")
print("=" * 70)
df_suppliers = pd.read_excel(file_path, sheet_name='供应商列表')
df_loss = pd.read_excel(file_path, sheet_name='转运商损耗率', index_col=0)
df_params = pd.read_excel(file_path, sheet_name='固定参数')
print("\n【1】从附件 1 统计历史最大供货量...")

df_supply_raw = pd.read_excel(supply_file, sheet_name='供应商的供货量
(m³)')
supply_cols = df_supply_raw.columns[2:]

real_max_supply = {}
for idx, row in df_supply_raw.iterrows():
    sid = row.iloc[0]
    max_val = row[supply_cols].max()
    real_max_supply[sid] = max_val

print(f" 已统计 {len(real_max_supply)} 家供应商的历史最大供货量")

selected = df_suppliers['供应商 ID'].tolist()
material_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['材料分类']))
mu_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['mu_i']))

max_supply_dict = {}

```

```

for sid in selected:
    if sid in real_max_supply and real_max_supply[sid] > 0:
        max_supply_dict[sid] = real_max_supply[sid]
    else:
        max_supply_dict[sid] = mu_dict.get(sid, 0) * 2
        if sid not in real_max_supply:
            print(f" 警告: {sid} 未在附件 1 中找到, 使用 mu_i*2")
        elif real_max_supply[sid] == 0:
            print(f" 警告: {sid} 历史最大供货量为 0, 使用 mu_i*2")

price_dict = {'A': 1.2, 'B': 1.1, 'C': 1.0}
r_m = {'A': 0.6, 'B': 0.66, 'C': 0.72}

loss_rate = df_loss.values

alpha_fixed = {'A': 0.35, 'B': 0.30, 'C': 0.35}
total_alpha = sum(alpha_fixed.values())
alpha_fixed = {k: v/total_alpha for k, v in alpha_fixed.items()}

P = df_params[df_params['参数'] == '周产能(m3)']['数值'].values[0]
D_m_t = {}
for m in ['A', 'B', 'C']:
    D_m_t[m] = [P * alpha_fixed[m] * r_m[m] for t in range(24)]

D_t = [sum(D_m_t[m][t] for m in ['A', 'B', 'C']) for t in range(24)]

I0 = df_params[df_params['参数'] == '初始库存(m3)']['数值'].values[0]
M = df_params[df_params['参数'] == '运力上限(m3)']['数值'].values[0]
T = int(df_params[df_params['参数'] == '规划周期(周)']['数值'].values[0])
I = int(df_params[df_params['参数'] == '供应商数']['数值'].values[0])
K = int(df_params[df_params['参数'] == '转运商数']['数值'].values[0])

print(f"\n 周产能: {P} m³")
print(f" 初始库存: {I0} m³")
print(f" 运力上限: {M} m³/周")
print(f" 供应商数: {I} 家")
print(f" 转运商数: {K} 家")
print(f" 规划周期: {T} 周")

M_big = max(max_supply_dict.values()) * 2
print(f" Big-M: {M_big:.2f}")

```

```

SAFETY_WEEKS = 2
avg_consumption = sum(D_t) / T
SAFETY_STOCK = SAFETY_WEEKS * avg_consumption

print(f"\n  平均周消耗: {avg_consumption:.2f} m³")
print(f"  安全库存线: {SAFETY_STOCK:.2f} m³")
print(f"  库存上限: {8 * avg_consumption:.2f} m³")

theta_A_lower = 0.24
theta_A_upper = 0.32
theta_C_lower = 0.28
theta_C_upper = 0.39

print(f"\n  【四边硬约束】 ")
print(f"    A 类占比: {theta_A_lower*100}% ~ {theta_A_upper*100}%")
print(f"    C 类占比: {theta_C_lower*100}% ~ {theta_C_upper*100}%")

print("\n 【2】 定义决策变量...")

model = pl.LpProblem("Final_Problem3_FourBounds", pl.LpMinimize)

x = pl.LpVariable.dicts("x", ((i, t) for i in range(I) for t in range(T)),
                        lowBound=0, cat='Continuous')

u = pl.LpVariable.dicts("u", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        cat='Binary')

w = pl.LpVariable.dicts("w", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        lowBound=0, cat='Continuous')

I_var = pl.LpVariable.dicts("I", (t for t in range(T)),
                            lowBound=0, cat='Continuous')

print(f"  x_{i,t}: {I*T} 个")
print(f"  u_{i,k,t}: {I*K*T} 个")
print(f"  w_{i,k,t}: {I*K*T} 个")
print(f"  I_t: {T} 个")

print("\n 【3】 构建加权目标函数...")

```

```

cost_purchase = pl.lpSum(
    price_dict[material_dict[selected[i]]] * x[(i, t)]
    for i in range(I) for t in range(T)
)

cost_inventory = pl.lpSum(0.05 * I_var[t] for t in range(T))

cost_loss = pl.lpSum(
    price_dict[material_dict[selected[i]]] * (loss_rate[k, t] / 100) * w[(i, k, t)]
    for i in range(I) for k in range(K) for t in range(T)
)

A_total = pl.lpSum(x[(i, t)] for i in range(I) for t in range(T)
    if material_dict[selected[i]] == 'A')
C_total = pl.lpSum(x[(i, t)] for i in range(I) for t in range(T)
    if material_dict[selected[i]] == 'C')
total_order = pl.lpSum(x[(i, t)] for i in range(I) for t in range(T))

f1 = cost_purchase + cost_inventory + cost_loss
f2 = A_total
f3 = C_total
f4 = cost_loss

f1_base = 501523.29
f2_max = 24.00
f3_base = 48.00
f4_base = 0.08

w4_fixed = 0.10
w1_original = 0.1502
w2_original = 0.1491
w3_original = 0.1491
total_original = w1_original + w2_original + w3_original

w1 = (w1_original / total_original) * (1 - w4_fixed)
w2 = (w2_original / total_original) * (1 - w4_fixed)
w3 = (w3_original / total_original) * (1 - w4_fixed)
w4 = w4_fixed

model += (
    w1 * (f1 / f1_base)
    - w2 * (f2 / (f2_max / 100))
)

```

```

        + w3 * (f3 / (f3_base / 100))
        + w4 * (f4 / (f4_base / 100))
    )

    print("  目标函数: 加权四目标 (修正权重) ")
    print(f"      w1(总成本)={w1:.4f}, w2(A 类)={w2:.4f}, w3(C 类)={w3:.4f},
w4(损耗率)={w4:.4f}")

    print("\n【4】添加约束条件")
    constraint_count = 0

    print("线性化约束")
    for i in range(I):
        for k in range(K):
            for t in range(T):
                model += w[(i, k, t)] <= M_big * u[(i, k, t)]
                model += w[(i, k, t)] <= x[(i, t)]
                model += w[(i, k, t)] >= x[(i, t)] - M_big * (1 - u[(i, k, t)])
                model += w[(i, k, t)] >= 0
                constraint_count += 4

    print("库存平衡方程")
    for t in range(T):
        receive = pl.lpSum((1 - loss_rate[k, t] / 100) * w[(i, k, t)]
                            for i in range(I) for k in range(K))

        if t == 0:
            model += I_var[t] == I0 + receive - D_t[t]
        else:
            model += I_var[t] == I_var[t-1] + receive - D_t[t]
        constraint_count += 1

    print("安全库存约束")
    for t in range(T):
        model += I_var[t] >= SAFETY_STOCK
        model += I_var[t] <= 8 * avg_consumption
        constraint_count += 2

    print("供应商能力约束")
    for i in range(I):
        max_supply = max_supply_dict[selected[i]]
        for t in range(T):
            model += x[(i, t)] <= max_supply

```

```

        constraint_count += 1

print("转运指派唯一性")
for i in range(I):
    for t in range(T):
        model += pl.lpSum(u[(i, k, t)] for k in range(K)) == 1
        constraint_count += 1

print("转运商运力约束")
for k in range(K):
    for t in range(T):
        model += pl.lpSum(w[(i, k, t)] for i in range(I)) <= M
        constraint_count += 1

print(" 四边硬约束")
model += A_total >= theta_A_lower * total_order
model += A_total <= theta_A_upper * total_order
model += C_total >= theta_C_lower * total_order
model += C_total <= theta_C_upper * total_order
constraint_count += 4

print(f"          {theta_A_lower*100}%    <=  A   类 占 比    <=
{theta_A_upper*100}%")
print(f"          {theta_C_lower*100}%    <=  C   类 占 比    <=
{theta_C_upper*100}%")

print(f"\n 共添加 {constraint_count} 个约束")

print("\n 【5】 求解 MILP 模型...")
print(" 求解器: PuLP (CBC)")
print(" 时间限制: 600 秒")
print(" 请稍候...")

model.solve(pl.PULP_CBC_CMD(msg=True, timeLimit=600, gapRel=0.01))

print("\n" + "=" * 70)
print(" 【求解结果】 ")
print("=" * 70)

status = pl.LpStatus[model.status]
print(f" 求解状态: {status}")

```

```

if model.status == 1:
    print(f'找到最优解! ')

    total_cost = pl.value(f1)
    A_val = pl.value(f2)
    C_val = pl.value(f3)
    total_val = pl.value(total_order)
    loss_val = pl.value(f4)

    A_ratio = A_val / total_val * 100 if total_val > 0 else 0
    C_ratio = C_val / total_val * 100 if total_val > 0 else 0
    loss_rate_total = loss_val / total_val * 100 if total_val > 0 else 0

    print(f'\n 总成本: {total_cost:.2f}')
    print(f' 采购成本: {pl.value(cost_purchase):.2f}')
    print(f' 库存成本: {pl.value(cost_inventory):.2f}')
    print(f' 损耗成本: {pl.value(cost_loss):.2f}')
    print(f'\n  【A/C 占比统计】 ')
    print(f'    A 类总订货量: {A_val:.2f} m³')
    print(f'    C 类总订货量: {C_val:.2f} m³')
    print(f'    总订货量: {total_val:.2f} m³')
    print(f'    A 类占比: {A_ratio:.2f}%')
    print(f'    C 类占比: {C_ratio:.2f}%')
    print(f'    总损耗率: {loss_rate_total:.2f}%')
    print(f'    加权目标值: {pl.value(model.objective):.6f}')

    print("\n  【硬约束检查】 ")
    print(f'    A 类占比 {A_ratio:.2f}% 在 [{theta_A_lower*100}%, {theta_A_upper*100}%] 内 : { if theta_A_lower*100 <= A_ratio <= theta_A_upper*100 else }')
    print(f'    C 类占比 {C_ratio:.2f}% 在 [{theta_C_lower*100}%, {theta_C_upper*100}%] 内 : { if theta_C_lower*100 <= C_ratio <= theta_C_upper*100 else }')

    with open("最终方案_四边约束_目标值.txt", 'w', encoding='utf-8') as f:
        f.write(f'总成本: {total_cost:.2f}\n')
        f.write(f'A 类占比: {A_ratio:.2f}%\n')
        f.write(f'C 类占比: {C_ratio:.2f}%\n')
        f.write(f'总损耗率: {loss_rate_total:.2f}%\n')
        f.write(f'A 类订货量: {A_val:.2f}\n')
        f.write(f'C 类订货量: {C_val:.2f}\n')
        f.write(f'总订货量: {total_val:.2f}\n')

```

```

        f.write(f'加权目标值: {pl.value(model.objective):.6f}\n')
        f.write(f'theta_A_lower: {theta_A_lower}\n')
        f.write(f'theta_A_upper: {theta_A_upper}\n')
        f.write(f'theta_C_lower: {theta_C_lower}\n')
        f.write(f'theta_C_upper: {theta_C_upper}\n')

print("\n 【6】 导出订购方案（附件 A 格式） ...")

all_supplier_ids = df_supply_raw.iloc[:, 0].tolist()

order_dict = {}
for i in range(I):
    sid = selected[i]
    order_dict[sid] = [pl.value(x[(i, t)]) for t in range(T)]

order_rows = []
for sid in all_supplier_ids:
    row = {'供应商 ID': sid}
    if sid in order_dict:
        for t in range(T):
            row[f'W {t+1}'] = order_dict[sid][t] if order_dict[sid][t] >
0.001 else 0
    else:
        for t in range(T):
            row[f'W {t+1}'] = 0
    order_rows.append(row)

df_order = pd.DataFrame(order_rows)
order_file = "附件 A_订购方案_最终.xlsx"
df_order.to_excel(order_file, index=False)
print(f' 已保存至: {order_file}')

print("\n 【7】 导出转运方案（附件 B 格式） ...")

trans_dict = {}
for i in range(I):
    sid = selected[i]
    trans_dict[sid] = {}
    for t in range(T):
        for k in range(K):
            trans_dict[sid][f'T {k+1}_W {t+1}'] = pl.value(w[(i, k, t)])

```

```

trans_rows = []
for sid in all_supplier_ids:
    row = {'供应商 ID': sid}
    if sid in trans_dict:
        for t in range(T):
            for k in range(K):
                key = f'T{k+1}_W{t+1}'
                val = trans_dict[sid].get(key, 0)
                row[key] = val if val > 0.001 else 0
    else:
        for t in range(T):
            for k in range(K):
                row[f'T{k+1}_W{t+1}'] = 0
    trans_rows.append(row)

df_trans = pd.DataFrame(trans_rows)
trans_file = "附件 B_转运方案_最终.xlsx"
df_trans.to_excel(trans_file, index=False)
print(f'  已保存至: {trans_file}')

print("\n【8】 导出详细订购与转运方案...")

results = []
for i in range(I):
    for t in range(T):
        x_val = pl.value(x[(i, t)])
        if x_val > 0.01:
            assigned_k = None
            for k in range(K):
                if pl.value(u[(i, k, t)]) > 0.5:
                    assigned_k = k
                    break
            if assigned_k is not None:
                results.append({
                    '周次': t+1,
                    '供应商': selected[i],
                    '材料': material_dict[selected[i]],
                    '订货量': x_val,
                    '转运商': f'T{assigned_k+1}',
                    '损耗率(%)': loss_rate[assigned_k, t]
                })

```

```

df_plan = pd.DataFrame(results)

inventory = []
for t in range(T):
    inventory.append({
        '周次': t+1,
        '期末库存': pl.value(I_var[t]),
        '本周消耗': D_t[t],
        '安全库存线': SAFETY_STOCK,
        '是否低于安全线': '是' if pl.value(I_var[t]) < SAFETY_STOCK
    else '否'
    })
df_inv = pd.DataFrame(inventory)

detail_file = "最终方案_四边约束_订购转运方案.xlsx"
with pd.ExcelWriter(detail_file, engine='openpyxl') as writer:
    df_plan.to_excel(writer, sheet_name='订购与转运方案',
index=False)
    df_inv.to_excel(writer, sheet_name='库存变化', index=False)

print(f" 已保存至: {detail_file}")

print("\n" + "=" * 70)
print(" 【最终方案关键指标汇总】 ")
print("=" * 70)
print(f"  A 类占比: {A_ratio:.2f}%")
print(f"  C 类占比: {C_ratio:.2f}%")
print(f"  总成本: {total_cost:.2f}")
print(f"  总损耗率: {loss_rate_total:.2f}%")
print(f"      A 类 占 比 范 围 : [{theta_A_lower*100}%,
{theta_A_upper*100}%]")
print(f"      C 类 占 比 范 围 : [{theta_C_lower*100}%,
{theta_C_upper*100}%]")
print("=" * 70)

else:
    print(f"求解失败！ 状态: {status}")
    model.writeLP("最终方案_四边约束_debug.lp")
    print(f"\n 已导出 最终方案_四边约束_debug.lp 文件")

print("\n" + "=" * 70)
print("最终方案求解完成！ ")

```

```
print("=" * 70)
```

问题四 MILP 求方案 python 代码

```
import pandas as pd
import pulp as pl
import warnings
warnings.filterwarnings('ignore')

print("=" * 70)
print("问题四 第 3 批: 以 P_max 代入 MILP 求解 24 周方案")
print("=" * 70)
df_suppliers = pd.read_excel(file_path, sheet_name='供应商列表')
df_loss = pd.read_excel(file_path, sheet_name='转运商损耗率', index_col=0)
df_params = pd.read_excel(file_path, sheet_name='固定参数')

with open(r"C:\Users\郭含梦\Desktop\数学建模论文\问题四_第 1 批结果.txt", 'r', encoding='utf-8') as f:
    results = {}
    for line in f:
        if ':' in line:
            key, val = line.strip().split(':', 1)
            results[key.strip()] = float(val.strip())

P_max = results['P_max']
print(f"\n 最大产能: {P_max:.2f} m³/周")

selected = df_suppliers['供应商 ID'].tolist()
material_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['材料分类']))
mu_dict = dict(zip(df_suppliers['供应商 ID'], df_suppliers['mu_i']))

print("\n 【1】 从附件 1 统计历史最大供货量...")
df_supply_raw = pd.read_excel(supply_file, sheet_name='供应商的供货量 (m³) ')
supply_cols = df_supply_raw.columns[2:]

real_max_supply = {}
for idx, row in df_supply_raw.iterrows():
    sid = row.iloc[0]
    max_val = row[supply_cols].max()
    real_max_supply[sid] = max_val
```

```

print(f" 已统计 {len(real_max_supply)} 家供应商的历史最大供货量")

max_supply_dict = {}
for sid in selected:
    if sid in real_max_supply and real_max_supply[sid] > 0:
        max_supply_dict[sid] = real_max_supply[sid]
    else:
        max_supply_dict[sid] = mu_dict.get(sid, 0) * 2

price_dict = {'A': 1.2, 'B': 1.1, 'C': 1.0}
r_m = {'A': 0.6, 'B': 0.66, 'C': 0.72}

loss_rate = df_loss.values

P0 = df_params[df_params['参数'] == '周产能(m3)']['数值'].values[0]
I0 = df_params[df_params['参数'] == '初始库存(m3)']['数值'].values[0]
M = df_params[df_params['参数'] == '运力上限(m3)']['数值'].values[0]
T = int(df_params[df_params['参数'] == '规划周期(周)']['数值'].values[0])
I = int(df_params[df_params['参数'] == '供应商数']['数值'].values[0])
K = int(df_params[df_params['参数'] == '转运商数']['数值'].values[0])

alpha_fixed = {'A': 0.35, 'B': 0.30, 'C': 0.35}
total_alpha = sum(alpha_fixed.values())
alpha_fixed = {k: v/total_alpha for k, v in alpha_fixed.items()}

D_m_t = {}
for m in ['A', 'B', 'C']:
    D_m_t[m] = [P_max * alpha_fixed[m] * r_m[m] for t in range(T)]

D_t = [sum(D_m_t[m][t] for m in ['A', 'B', 'C']) for t in range(T)]

avg_consumption = sum(D_t) / T
SAFETY_STOCK = 2 * avg_consumption

print(f"\n 新产能下的参数:")
print(f" 每周产能: {P_max:.2f} m³/周")
print(f" 平均周消耗: {avg_consumption:.2f} m³")
print(f" 安全库存线: {SAFETY_STOCK:.2f} m³")

print("\n 构建 MILP 模型...")
model = pl.LpProblem("Problem4_24Weeks", pl.LpMinimize)

```

```

x = pl.LpVariable.dicts("x", ((i, t) for i in range(I) for t in range(T)),
                        lowBound=0, cat='Continuous')

u = pl.LpVariable.dicts("u", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        cat='Binary')

w = pl.LpVariable.dicts("w", ((i, k, t) for i in range(I) for k in range(K) for t in
range(T)),
                        lowBound=0, cat='Continuous')

I_var = pl.LpVariable.dicts("I", (t for t in range(T)), lowBound=0,
cat='Continuous')

cost_purchase = pl.lpSum(price_dict[material_dict[selected[i]]] * x[(i, t)]
                        for i in range(I) for t in range(T))
cost_inventory = pl.lpSum(0.05 * I_var[t] for t in range(T))
cost_loss = pl.lpSum(price_dict[material_dict[selected[i]]] * (loss_rate[k, t] /
100) * w[(i, k, t)]
                        for i in range(I) for k in range(K) for t in range(T))
model += cost_purchase + cost_inventory + cost_loss

print("添加约束...")
M_big = max(max_supply_dict.values()) * 2

for i in range(I):
    for k in range(K):
        for t in range(T):
            model += w[(i, k, t)] <= M_big * u[(i, k, t)]
            model += w[(i, k, t)] <= x[(i, t)]
            model += w[(i, k, t)] >= x[(i, t)] - M_big * (1 - u[(i, k, t)])
            model += w[(i, k, t)] >= 0

for t in range(T):
    receive = pl.lpSum((1 - loss_rate[k, t] / 100) * w[(i, k, t)]
                        for i in range(I) for k in range(K))

    if t == 0:
        model += I_var[t] == I0 + receive - D_t[t]
    else:
        model += I_var[t] == I_var[t-1] + receive - D_t[t]

for t in range(T):
    model += I_var[t] >= SAFETY_STOCK
    model += I_var[t] <= 8 * avg_consumption

```

```

for i in range(I):
    max_supply = max_supply_dict.get(selected[i], 0)
    if max_supply > 0:
        for t in range(T):
            model += x[(i, t)] <= max_supply

for i in range(I):
    for t in range(T):
        model += pl.lpSum(u[(i, k, t)] for k in range(K)) == 1

for k in range(K):
    for t in range(T):
        model += pl.lpSum(w[(i, k, t)] for i in range(I)) <= M

print("求解中...")
model.solve(pl.PULP_CBC_CMD(msg=False, timeLimit=300, gapRel=0.01))

if model.status == 1:
    total_cost = pl.value(model.objective)
    total_order = sum(pl.value(x[(i, t)]) or 0 for i in range(I) for t in range(T))

    print(f"\n 总成本: {total_cost:.2f}")
    print(f'总订货量: {total_order:.2f} m³')

    print("\n 【导出附件 A: 订购方案】 ")

    all_supplier_ids = df_supply_raw.iloc[:, 0].tolist()

    order_dict = {}
    for i in range(I):
        sid = selected[i]
        order_dict[sid] = [pl.value(x[(i, t)]) or 0 for t in range(T)]

    order_rows = []
    for sid in all_supplier_ids:
        row = {'供应商 ID': sid}
        if sid in order_dict:
            for t in range(T):
                row[f'W {t+1}'] = order_dict[sid][t] if order_dict[sid][t] >
0.001 else 0
            else:
                for t in range(T):

```

```

        row[f'W {t+1}'] = 0
        order_rows.append(row)

df_order = pd.DataFrame(order_rows)
order_file = r"C:\Users\郭含梦\Desktop\数学建模论文\问题四_附件 A_
订购方案.xlsx"
df_order.to_excel(order_file, index=False)
print(f"  已保存至: {order_file}")

print("\n 【导出附件 B: 转运方案】 ")

trans_dict = {}
for i in range(I):
    sid = selected[i]
    trans_dict[sid] = {}
    for t in range(T):
        for k in range(K):
            trans_dict[sid][f'T {k+1}_W {t+1}'] = pl.value(w[(i, k, t)])
or 0

trans_rows = []
for sid in all_supplier_ids:
    row = {'供应商 ID': sid}
    if sid in trans_dict:
        for t in range(T):
            for k in range(K):
                key = f'T {k+1}_W {t+1}'
                val = trans_dict[sid].get(key, 0)
                row[key] = val if val > 0.001 else 0
    else:
        for t in range(T):
            for k in range(K):
                row[f'T {k+1}_W {t+1}'] = 0
    trans_rows.append(row)

df_trans = pd.DataFrame(trans_rows)
trans_file = r"C:\Users\郭含梦\Desktop\数学建模论文\问题四_附件 B_
转运方案.xlsx"
df_trans.to_excel(trans_file, index=False)
print(f"  已保存至: {trans_file}")

print("\n 【导出详细订购与转运方案】 ")

```

```

results_detail = []
for i in range(I):
    for t in range(T):
        x_val = pl.value(x[(i, t)]) or 0
        if x_val > 0.01:
            assigned_k = None
            for k in range(K):
                if pl.value(u[(i, k, t)]) > 0.5:
                    assigned_k = k
                    break
            if assigned_k is not None:
                results_detail.append({
                    '周次': t+1,
                    '供应商': selected[i],
                    '材料': material_dict[selected[i]],
                    '订货量': x_val,
                    '转运商': f'T{assigned_k+1}'
                })

df_plan = pd.DataFrame(results_detail)

inventory = []
for t in range(T):
    inventory.append({
        '周次': t+1,
        '期末库存': pl.value(I_var[t]),
        '本周消耗': D_t[t],
        '安全库存线': SAFETY_STOCK,
        '是否低于安全线': '是' if pl.value(I_var[t]) < SAFETY_STOCK
    })
else '否'

df_inv = pd.DataFrame(inventory)

detail_file = r"C:\Users\郭含梦\Desktop\数学建模论文\问题四_订购转运方案_详细.xlsx"
with pd.ExcelWriter(detail_file, engine='openpyxl') as writer:
    df_plan.to_excel(writer, sheet_name='订购与转运方案',
index=False)
    df_inv.to_excel(writer, sheet_name='库存变化', index=False)

print(f"  已保存至: {detail_file}")

```

```
print("\n" + "=" * 70)
print(" 【问题四第 3 批完成!】 ")
print("=" * 70)
print(f" 附件 A: {order_file}")
print(f" 附件 B: {trans_file}")
print(f" 详细方案: {detail_file}")
print("=" * 70)

else:
    print(f"MILP 求解失败, 状态: {p1.LpStatus[model.status]}")
    model.writeLP("问题四_debug.lp")
```